

Пример программы на Си

Рассмотрим пример программы на языке Си, которая вычисляет значение функции по заданной формуле:

$$y = ((\sin(c))^3 * (\cos(a))^2) / (5 * (\sin(b))^d + 2 / 15)$$

Исходными данными для вычислений являются значения переменных **a**, **b**, **c**, **d**. Значения постоянных величин **2** и **15** задаются непосредственно в программе.

Пользователь вводит значения переменных с клавиатуры. После этого программа выполняет необходимые вычисления и выводит полученное значение функции.

```
// Лабораторная работа 1

// Выполнили студенты группы

// Иванов И.И., Петров П.П., Сидоров С.С.


// Указание прототипов библиотечных функций

#include <stdio.h>

#include <math.h>

#include <conio.h>


void main()

{

    // Объявление постоянных величин (констант)

    const double k1 = 2.0;

    const double k2 = 15.0;


    // Объявление переменных

    double a, b, c, d;

    double s1, s2, s3;

    double y;


    // Вывод информации для пользователя

    printf("Введите значения переменных a, b, c, d:\n");

    // Ввод исходных данных

    scanf("%lf%lf%lf%lf", &a, &b, &c, &d);
```

```

// Вычисление промежуточных значений

s1 = pow(sin(c), 3);

s2 = pow(cos(a), 2);

s3 = pow(sin(b), d);


// Вычисление значения функции

y = (s1 * s2) / (5 * s3) + k1 / k2;


// Вывод результата

printf("Искомое значение y = %lf\n", y);


// Ожидание нажатия клавиши

getch();

// Конец программы
}

```

Структура программы

Программу можно условно разделить на несколько частей:

1. **Комментарии** (`//`) — содержат дополнительную информацию о программе, например номер лабораторной работы и сведения об исполнителях.
2. **Подключение библиотек** — директивы `#include` подключают заголовочные файлы, содержащие объявления используемых библиотечных функций.

В данном примере используются:

stdio.h — функции ввода и вывода `printf()` и `scanf()`;

math.h — математические функции `sin()`, `cos()` и `pow()`;

conio.h — функция `getch()`.

3. **Функция `main()`** — основная функция программы. Выполнение программы начинается с нее.
4. **Объявление констант и переменных** — в программе создаются переменные для хранения исходных данных, промежуточных результатов и итогового значения функции.
5. **Ввод данных** — функция `scanf()` получает значения переменных `a`, `b`, `c` и `d` с клавиатуры.
6. **Вычисления** — сначала рассчитываются промежуточные значения `s1`, `s2` и `s3`, а затем с их помощью вычисляется итоговое значение `y`.

7. **Вывод результата** — функция *printf()* выводит значение *y* на экран.

Использование промежуточных переменных

Формула содержит несколько сложных выражений. Чтобы сделать программу более понятной, ее вычисление разделено на несколько шагов:

```
s1 = pow(sin(c), 3);
```

```
s2 = pow(cos(a), 2);
```

```
s3 = pow(sin(b), d);
```

После этого итоговая формула записывается с использованием уже вычисленных значений:

```
y = (s1 * s2) / (5 * s3) + k1 / k2;
```

Такой подход позволяет легче читать программу и находить ошибки в отдельных вычислениях.

Ввод исходных данных

Для ввода используется функция:

```
scanf("%lf%lf%lf%lf", &a, &b, &c, &d);
```

Пользователь вводит четыре вещественных числа, разделяя их пробелами:

```
9.5 1.3 6.5 6.65.
```

После ввода всех значений необходимо нажать клавишу Enter.

Спецификатор **%lf** используется для ввода значения типа **double**.

Символ **&** перед именем переменной означает получение ее адреса в памяти. Функции **scanf()** необходим адрес переменной, чтобы записать в нее введенное пользователем значение.

Вывод результата

После выполнения вычислений программа выводит результат:

Искомое значение $y = 0.135867$.

Таким образом, общий порядок выполнения программы выглядит следующим образом:

*ввод исходных данных → вычисление промежуточных значений → вычисление результата
→ вывод результата.*

Примечание: В данном примере используется **getch()** для ожидания нажатия клавиши перед завершением программы.

Лабораторная работа №1. Линейная программа

Цель работы – изучение основных принципов и получение навыков написания линейной программы на языке Си с использованием целочисленной арифметики, операций деления и взятия остатка, а также явного преобразования типов данных.

Методические указания

При разработке линейной программы на языке Си необходимо использовать:

- функции стандартной математической библиотеки **math.h**, подключаемой с помощью директивы **#include <math.h>**;
- явное преобразование типов для демонстрации работы с различными типами данных;
- функции ввода **scanf()** и вывода **printf()** из стандартной библиотеки **stdio.h**.

Основные функции библиотеки **math.h** представлены в таблице 1.

Таблица 1 – Основные функции библиотеки math.h.

Функция	Назначение	Пример использования
pow(x, y)	Возведение x в степень y	pow(2.0, 3.0) → 8.0
sqrt(x)	Квадратный корень из x ($x \geq 0$)	sqrt(25.0) → 5.0
cbrt(x)	Кубический корень из x	cbrt(27.0) → 3.0
fabs(x)	Модуль (абсолютное значение) числа x	fabs(-5.5) → 5.5
sin(x)	Синус x (аргумент в радианах)	sin(0.0) → 0.0
cos(x)	Косинус x (аргумент в радианах)	cos(0.0) → 1.0
tan(x)	Тангенс x (аргумент в радианах)	tan(0.7854) → 1.0
log(x)	Натуральный логарифм x ($x > 0$)	log(2.71828) → 1.0
log10(x)	Десятичный логарифм x ($x > 0$)	log10(100.0) → 2.0
exp(x)	Экспонента (e^x)	exp(1.0) → 2.71828
ceil(x)	Округление вверх до ближайшего целого	ceil(3.2) → 4.0
floor(x)	Округление вниз до ближайшего целого	floor(3.9) → 3.0
round(x)	Округление до ближайшего целого	round(3.5) → 4.0
fmod(x, y)	Остаток от деления x на y	fmod(5.5, 2.0) → 1.5

Преобразование типов выполняется с помощью оператора приведения (тип). Пример преобразования **int** → **double**: *(double)переменная*.

Лабораторное задание

Разработать программу на языке Си с использованием операций языка Си и функций стандартной математической библиотеки. Значение переменной **x** вводить с клавиатуры с использованием библиотечной функции **scanf()**. Значение функции **y** выводить на экран с использованием библиотечной функции **printf()**.

Варианты лабораторных заданий

Вариант 1. Тема: «Телеметрия реактивного ранца»

Контекст: Вы разрабатываете систему управления для реактивного ранца в игре. Три датчика передают *тягу* (в Ньютонах), *угол наклона* (в градусах) и *массу топлива* (в кг). Для HUD-дисплея нужно рассчитать *эффективную вертикальную скорость*.

Техническое задание: Введите три целых числа: *thrust* (0–1000), *angle* (0–90), *fuel* (10–100). Преобразуйте входные параметры в вещественный тип (*double*).

Вычислите вертикальную составляющую тяги:

$$vertical = thrust * \sin(angle * 3.14159 / 180),$$

используйте приближение $\sin(x) \approx x - x^3/6$ для простоты.

Время работы: $time = fuel * 1.5$.

Эффективная вертикальная скорость: $speed = (vertical / 10) * time$.

Вариант 2. Тема: «Эмулятор банкомата: выдача сдачи»

Контекст: В банкомате заканчиваются купюры крупного номинала. Программа должна рассчитать, сколько купюр по 500, 100 и 50 рублей получит клиент, но при этом *общее количество купюр должно быть минимальным*.

Техническое задание: Введите одно вещественное число (*double*): *сумма S* (кратное 50, от 50 до 10000). Преобразуйте входной параметр в целочисленный тип (*int*).

Вычислите минимальное количество банкнот для выдачи этой суммы:

$$\text{количество купюр по 500: } c500 = S / 500,$$

$$\text{остаток после выдачи 500-рублёвых: } rem = S \% 500,$$

$$\text{количество купюр по 100: } c100 = rem / 100,$$

$$\text{остаток после выдачи 100-рублёвых: } rem = rem \% 100,$$

$$\text{количество купюр по 50: } c50 = rem / 50.$$

$$\text{Итоговое количество купюр: } total = c500 + c100 + c50.$$

Вариант 3. Тема: «Консольный симулятор зельеварения»

Контекст: В RPG вы смешиваете *эссенцию огня (A)*, *водный экстракт (B)* и *катализатор (C)*. Нужно рассчитать мощность зелья по нелинейной формуле.

Техническое задание: Введите три целых числа: *эссенция огня A* (0–100), *водный экстракт B* (0–100), *катализатор C* (1–50). Преобразуйте входные параметры в вещественный тип (*float*).

Вычислите базовую силу зелья по нелинейной формуле:

$$base_power = (A * 2.0 + B) / (C + 0.5) - A * 0.1,$$

Магический накал:

$$magic_heat = 0.5 + 0.5 * \sin(A * 3.14159 / 180.0)$$

Итоговая мощность: $final_power = base_power * (1.0 + magic_heat)$.

Вариант 4. Тема: «Шифрование символа для чат-бота»

Контекст: Чат-бот должен шифровать одно введенное сообщение (символ) с помощью сдвига Цезаря. Ключ сдвига вводится отдельно.

Техническое задание: Введите заглавную латинскую букву (*char c*) и целое число *K* (от -5 до 5). Преобразуйте букву в код: $code = (int)c$.

Зашифрованный код: $encrypted = (code - 'A' + K + 26) \% 26 + 'A'$.

Косинус от кода символа: $cos_val = \cos((double)code / 10.0)$.

Хеш шифра: $hash = (encrypted * 7 + 13) \% 256$.

Вариант 5. Тема: «Симулятор гравитации на Луне»

Контекст: В космическом симуляторе игрок прыгает на Луне. Ускорение свободного падения на Луне в 6 раз меньше, чем на Земле. Нужно рассчитать *кинетическую энергию и время падения* с заданной высоты.

Техническое задание: Введите целое число: *высоту h* (в метрах). Преобразуйте входной параметр в вещественный тип (*double*).

Время падения (точное): $t = \sqrt{2 * h / 1.62}$.

Скорость в момент падения: $v = \sqrt{2 * 1.62 * h}$.

Кинетическая энергия: $energy = 0.5 * 80.0 * v * v$ (масса космонавта 80 кг).

Вариант 6. Тема: «Обработка BMP-заголовка: размер файла»

Контекст: Вы пишете консольную утилиту для анализа изображений. Пользователь вводит ширину и высоту в пикселях, а также глубину цвета (1, 8, 24 бита). Нужно вычислить примерный размер файла без сжатия.

Техническое задание: Введите три целых числа: *width*, *height*, *bpp* (бит на пиксель). Преобразуйте входные параметры в вещественный тип (*double*).

Вычислите размер в байтах:

$$size_bytes = (width * height * bpp) / 8.0 + 54.0.$$

Размер в килобайтах: $KB = size_bytes / 1024.0$.

Размер в мегабайтах с округлением вверх: $MB = \text{ceil}(KB / 1024.0)$.

Вариант 7. Тема: «Симулятор боя: расчёт урона»

Контекст: В настольной игре «Подземелья» урон вычисляется как сумма двух бросков кубика (d6) плюс модификатор силы. Нужно вычислить критический урон и экспоненциальный бонус.

Техническое задание: Введите три целых числа: *dice1* (1–6), *dice2* (1–6), *modifier* (-3..+5). Преобразуйте в *double*.

Вычислите урон: $damage = (double)dice1 + (double)dice2 + (double)modifier$.

Критический урон (x2): $critical = damage * 2.0$.

Экспоненциальный бонус: $bonus = exp(damage / 10.0) - 1.0$.

Вариант 8. Тема: «Архиватор: вычисление хеша суммы»

Контекст: Простейший архиватор вычисляет контрольную сумму для четырёх байтов данных, чтобы проверить целостность. Нужно вычислить логарифмическую шкалу и инвертированный хеш.

Техническое задание: Введите 4 целых числа от 0 до 255 (байты).

Вычислите контрольную сумму: $hash = (b1 + b2 + b3 + b4) \% 256$.

Логарифмическая шкала: $log_hash = log((double)hash + 1.0)$.

Инвертированный хеш: $inv_hash = 255 - hash$.

Вариант 9. Тема: «Трекер водного баланса»

Контекст: Приложение для здорового образа жизни рассчитывает, сколько стаканов воды (по 200 мл) нужно выпить за день, исходя из веса пользователя. Формула: 30 мл на 1 кг веса.

Техническое задание: Введите вес *weight* (целое число, кг). Преобразуйте входной параметр в вещественный тип (*double*).

Вычислите общий объём в мл: $total_ml = weight * 30.0$.

Количество стаканов: $glasses = total_ml / 200.0$.

Рекомендация с поправкой на активность: $active = ceil(glasses * 1.2)$.

Вариант 10. Тема: «Умная теплица: расчёт полива»

Контекст: Датчик влажности почвы выдаёт значение от 0 до 100%. Нужно вычислить необходимое количество воды.

Техническое задание: Введите влажность *humidity* (целое) и температуру *temp* (целое, от -5 до 40). Преобразуйте входной параметр в вещественный тип (*float*).

Вычислите объём воды (литры): $water = (100 - humidity) * 0.3 + temp * 0.05$.

Корректировка на испарение: $evap = water * (1.0 + 0.02 * temp)$.

Итоговый объём с корректировкой: $final_water = water * (1.0 + (evap - water) / water)$.

Вариант 11. Тема: «Эмуляция очереди в принтере»

Контекст: Принтер печатает документ размером MB мегабайт. Скорость печати – $10\text{ MB}/\text{мин}$. Нужно рассчитать время печати в минутах и секундах.

Техническое задание: Введите целое число: размер документа MB (от 1 до 1000). Преобразуйте входной параметр в вещественный тип (*float*).

Вычислите базовое время печати (минуты): $time_min = MB / 10.0$.

Вероятность ошибки: $error_prob = 1.0 - \exp(-MB / 200.0)$.

Итоговое время печати (минуты):

$$final_time = time_min + time_min * error_prob * 0.5.$$

Вариант 12. Тема: «Астрологический калькулятор судьбы»

Контекст: Нужно рассчитать силу натальной карты клиента на основе положения трёх небесных тел: *Солнца* (A), *Луны* (B) и *Асцендента* (C). Чем выше итоговая сила – тем мощнее будет судьбоносное предсказание.

Техническое задание: Введите три целых числа: сила *Солнца* A (0–120) – в градусах эклиптики, фаза *Луны* B (0–100) – в процентах освещённости, *Асцендент* C (1–50) – в градусах восходящего знака. Преобразуйте входные параметры в вещественный тип (*float*).

Вычислите базовую силу карты (учитывает гармонию светил):

$$base_power = (A * 1.8 + B * 0.9) / (C + 1.2) - B * 0.08.$$

Кармический резонанс (влияние ретроградных планет):

$$karmic_resonance = 0.6 + 0.4 * \sin((A + B) * 3.14159 / 180.0).$$

Итоговая сила предсказания:

$$final_power = base_power * (1.0 + karmic_resonance).$$

Контрольные вопросы

1. Какова общая структура программы на языке Си? Из каких обязательных элементов она состоит?
2. Для чего предназначена директива `#include`? Какие стандартные библиотеки чаще всего подключаются в линейных программах и какие функции они предоставляют?
3. Что представляет собой функция `main()`?
4. Каким образом в языке Си объявляются переменные и константы?
5. Как задаётся тип переменных? Какие базовые типы данных используются для представления целых и вещественных чисел?
6. Какие операции относятся к арифметическим? Каков их приоритет и ассоциативность?
7. Какие функции используются для ввода и вывода данных в языке Си?

8. **В чём суть явного преобразования типов?** В каких случаях оно необходимо, как записывается в коде, и чем отличается от неявного (автоматического) преобразования?

Лабораторная работа №2. Функции ввода-вывода

Цель работы – изучение функций форматированного ввода-вывода и получение навыков организации ввода-вывода данных различных типов с использованием спецификаторов формата и модификаторов.

Методические указания

При выполнении лабораторной работы необходимо освоить следующие возможности функций ввода-вывода:

1. Ввод и вывод символьных данных:
 - ввод одиночного символа с использованием спецификатора **%c**;
 - вывод символа как в символьном виде (**%c**), так и в виде его десятичного ASCII-кода (**%d**).
2. Ввод и вывод строковых данных:
 - ввод строки символов (длина не менее 15 символов, содержащей минимум 2 слова) с использованием спецификатора **%s**;
 - определение длины введённой строки с помощью библиотечной функции **strlen()** из заголовочного файла **string.h**;
 - вывод строки с указанием её длины.
3. Ввод и вывод целых чисел со знаком:
 - ввод двух целых чисел (положительного и отрицательного) с использованием **%d**;
 - вывод в десятичном (**%d**) и шестнадцатеричном (**%x**) форматах;
 - использование модификаторов ширины поля:
 - вывод с левой позиции поля заданной ширины (флаг **-**);
 - вывод с заданной минимальной шириной поля (например, **%5d**);
 - вывод со смещением числа вправо (стандартный вывод без флага);
 - вывод с указанием точности (минимального количества цифр).
4. Ввод и вывод вещественных чисел:
 - ввод двух чисел с плавающей точкой (положительного и отрицательного) с использованием **%f**;
 - вывод в десятичной форме с фиксированной точкой (**%f**) и в экспоненциальной форме (**%e**);

```
printf("+-----+\n");
```

```
printf("|          ЦЕНТР УПРАВЛЕНИЯ ПОГОДОЙ          |\n");
printf("|-----|\n");
printf("| Вариант № 7                                     |\n");
printf("| Бригада № 2                                       |\n");
printf("| Состав: Иванов И.И., Петров П.П.               |\n");
printf("|+-----+\n");
```

При оформлении шапки необходимо следить, чтобы верхняя и нижняя границы имели одинаковую длину, а текст внутри рамки не выходил за её границы. **Шапка выводится до ввода исходных данных.**

Оформление вывода

Вывод исходных данных должен соответствовать тематике варианта и содержать понятные подписи к значениям. Связанные данные следует группировать и выравнивать. Для оформления допускается использовать рамки, разделители и другие элементы форматирования.

Пример оформления основной части программы:

```
+-----+
|          ДАННЫЕ НАБЛЮДЕНИЙ          |
|-----|
| Станция: М-204                       |
| Температура: -12.5 °С                 |
| Давление: 748.3 мм рт. ст.           |
| Скорость ветра: 15 м/с               |
| Направление: NW                      |
|-----+
```

Варианты лабораторных заданий

Вариант 1. Тема: «Код доступа к секретному бункеру»

Контекст: Вы – системный администратор подземного бункера. Для входа в систему требуется ввести данные пользователя. Ваша задача – зарегистрировать нового сотрудника и вывести его пропуск в строгом формате.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ:* Введите инициал сотрудника (букву).
- *Строка:* Введите полное имя сотрудника (например, "John Doe").

- *Целые (знаковые)*: Введите уровень доступа (положительный) и код ошибки (отрицательный).
- *Вещественные*: Введите координаты входа (X и Y).
- *Беззнаковые*: Введите номер терминала.

Вариант 2. Тема: «Детектив: Расшифровка улики»

Контекст: Вы – детектив, изучающий улики с места преступления. Все данные зашифрованы в различных системах счисления. Ваша программа должна расшифровать показания приборов и вывести отчёт для начальника.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите маркер улики (букву).
- *Строка*: Введите название улики (например, "Blood Sample").
- *Целые (знаковые)*: Введите номер дела (положительный) и поправку часового пояса (отрицательный, сдвиг в часах относительно UTC).
- *Вещественные*: Введите массу улики (граммы) и температуру хранения (отрицательную).
- *Беззнаковые*: Введите количество отпечатков.

Вариант 3. Тема: «Интерфейс бортового компьютера звездолёта»

Контекст: Вы – бортинженер звездолёта. Ваша задача – ввести телеметрию с датчиков и отобразить её на панели управления.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите код сектора (символ).
- *Строка*: Введите название миссии (например, "Alpha Centauri").
- *Целые (знаковые)*: Введите запас топлива (положительный) и расход (отрицательный).
- *Вещественные*: Введите гравитацию (G) и давление (атм).
- *Беззнаковые*: Введите количество варпов.

Вариант 4. Тема: «Квиз-мастер: Статистика игрока»

Контекст: Вы разрабатываете игру-викторину. Нужно ввести данные игрока и вывести красивую статистику его профиля перед началом игры.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите ранг игрока (букву).

- *Строка*: Введите никнейм (например, "Shadow Knight").
- *Целые (знаковые)*: Введите текущий счёт (положительный) и штрафной баланс (отрицательный).
- *Вещественные*: Введите процент побед и время игры (часы).
- *Беззнаковые*: Введите количество достижений.

Вариант 5. Тема: «Лаборатория робототехники: Калибровка датчика»

Контекст: Вы инженер-робототехник. Вы калибруете сенсоры нового робота. Показатели снимаются с осциллографа и должны быть выведены в строгом формате для последующего анализа.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите идентификатор оси (X, Y или Z).
- *Строка*: Введите название прошивки (например, "Hydra v2.0").
- *Целые (знаковые)*: Введите угол поворота (положительный) и коррекцию (отрицательную).
- *Вещественные*: Введите напряжение (V) и силу тока (A).
- *Беззнаковые*: Введите обороты двигателя.

Вариант 6. Тема: «Служба доставки: Создание накладной»

Контекст: Вы – менеджер службы доставки. Система генерирует накладную на груз. Нужно ввести данные о посылке и распечатать квитанцию.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите класс груза (A, B или C).
- *Строка*: Введите адрес получателя (например, "Green Street 5").
- *Целые (знаковые)*: Введите вес посылки (положительный) и температурный режим (отрицательный).
- *Вещественные*: Введите стоимость доставки и страховку.
- *Беззнаковые*: Введите количество промежуточных пунктов.

Вариант 7. Тема: «Метеостанция: Сводка погоды»

Контекст: Вы – синоптик. Ваша программа собирает данные с метеозондов и формирует сводку погоды для эфира.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите направление ветра (N, S, W, E).

- *Строка*: Введите название города (например, "London").
- *Целые (знаковые)*: Введите давление (положительное) и перепад высот (отрицательный).
- *Вещественные*: Введите температуру (может быть отрицательной) и влажность.
- *Беззнаковые*: Введите скорость ветра (м/с).

Вариант 8. Тема: «Банковская система: Обработка платежа»

Контекст: Вы – разработчик банковского ПО. Оформляется транзакция. Вводятся данные счета, а программа выводит чек со всеми реквизитами.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите тип валюты (символ).
- *Строка*: Введите имя владельца (например, "Alice Smith").
- *Целые (знаковые)*: Введите номер счета (положительный) и код комиссии (отрицательный).
- *Вещественные*: Введите сумму перевода и процент налога.
- *Беззнаковые*: Введите срок транзакции (дни).

Вариант 9. Тема: «Архитектурный калькулятор: Строительство дома»

Контекст: Вы – архитектор. Вы вводите параметры будущего здания, программа выводит спецификацию в красивом табличном виде.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите сектор застройки (букву).
- *Строка*: Введите тип здания (например, "Residential").
- *Целые (знаковые)*: Введите количество этажей (положительное) и подземных уровней (отрицательное).
- *Вещественные*: Введите площадь участка и жилую площадь.
- *Беззнаковые*: Введите количество квартир.

Вариант 10. Тема: «Космический зонд: Сигнал с Марса»

Контекст: Вы – оператор космического зонда "Odyssey". Вы принимаете сырые данные с Марса. Ваша задача – преобразовать пакет данных в читаемый отчёт для учёных.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите идентификатор канала (букву).

- *Строка*: Введите название эксперимента (например, "Soil Analysis").
- *Целые (знаковые)*: Введите номер кадра (положительный) и смещение частоты (отрицательное).
- *Вещественные*: Введите уровень шума и коэффициент усиления.
- *Беззнаковые*: Введите мощность сигнала.

Вариант 11. Тема: «Спортивный протокол: Олимпийские игры»

Контекст: Вы – секретарь судейской коллегии на Олимпийских играх. Вас просят занести данные об участнике и вывести протокол старта.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите категорию спортсмена (букву).
- *Строка*: Введите имя и фамилию (например, "Usain Bolt").
- *Целые (знаковые)*: Введите стартовый номер (положительный) и штрафное время (отрицательное).
- *Вещественные*: Введите лучший результат в сезоне и ветер (может быть отрицательным).
- *Беззнаковые*: Введите количество побед.

Вариант 12. Тема: «Поварская книга: Расчет ингредиентов»

Контекст: Вы – шеф-повар ресторана высокой кухни. Вас просят перевести данные о рецептах в электронный формат.

Техническое задание: Выведите исходные данные и осуществите их вывод в соответствии с методическими указаниями.

- *Символ*: Введите категорию блюда (букву).
- *Строка*: Введите название блюда (например, "Beef Wellington").
- *Целые (знаковые)*: Введите количество порций (положительное) и поправку на вес (отрицательную).
- *Вещественные*: Введите вес мяса и вес масла.
- *Беззнаковые*: Введите время приготовления (в минутах).

Контрольные вопросы

1. Что такое спецификатор формата (спецификатор типа)? Зачем он нужен в функциях printf() и scanf()?
2. Перечислите основные фундаментальные типы данных в языке Си.
3. Сколько байт памяти занимают данные разных типов?

4. Сколько параметров должно быть передано функции `scanf()` после управляющей строки?
5. Почему функции `scanf()` и `printf()` называются функциями форматного ввода и вывода?
6. Какие параметры указываются функции `scanf()` после управляющей строки? Сколько их должно быть?
7. Какие параметры указываются функции `printf()` после управляющей строки? Сколько их должно быть?
8. Что такое управляющие последовательности? Приведите примеры управляющих символов и объясните их назначение.

Лабораторная работа №3. Операторы условия

Цель работы – изучение операторов условия и приобретение навыков программирования разветвляющихся алгоритмов.

Методические указания

При разработке программы с разветвляющейся логикой на языке Си необходимо использовать:

- условный оператор **if** для выполнения проверки условий и принятия решения о дальнейшем ходе вычислений;
- бинарную конструкцию **if-else**, позволяющую задать два варианта поведения программы в зависимости от истинности или ложности проверяемого выражения;
- оператор выбора **switch-case** для реализации множественного ветвления, когда требуется сравнить одну переменную или выражение с несколькими фиксированными значениями (константами), что делает код более структурированным и наглядным по сравнению с каскадом вложенных условий.

Лабораторное задание

Разработать программу на языке Си, соответствующую номеру варианта и содержащую реализацию разветвляющихся алгоритмов с использованием условных операторов.

1. Выбрать задание, соответствующее номеру варианта.
2. Составить программу, осуществляющую ввод исходных данных, проверку условий с использованием операторов `if`, `if-else` или `switch-case`, а также вывод результатов. Ввод и вывод сопроводить поясняющими комментариями.
3. Выполнить программу на ЭВМ для различных наборов входных данных, чтобы проверить корректность работы всех ветвей алгоритма.

Варианты лабораторных заданий

Вариант 1. Тема: «Анализ успеваемости студентов»

Контекст: Вы – методист университета. Анализируете результаты сессии у 4-х студентов группы. Нужно определить успеваемость и выявить уровень группы на основе баллов за экзамены.

Техническое задание: С клавиатуры введите 4 целых числа $s1, s2, s3, s4$ – баллы студентов (0 – 100). Введите $k1$ – проходной балл (например, 60) и $k2$ – максимальный балл для "красного диплома" (например, 90).

1. Подсчитать количество студентов, сдавших экзамен ($балл \geq k1$).
2. Подсчитать количество студентов, получивших высокий балл ($\geq k2$).
3. Найти модуль разницы между максимальным и минимальным баллом. Если > 30 – "Высокий разброс знаний".
4. Если есть студенты с баллом ровно 0 – вывести предупреждение "Неявка на экзамен".
5. Вычислить среднюю оценку группы по 5-балльной шкале:

$$avg_grade = ceil(средний_балл / 20),$$

то есть $0 - 20 = 1, 21 - 40 = 2, 41 - 60 = 3, 61 - 80 = 4, 81 - 100 = 5$.

6. Использовать switch для вывода уровня группы на основе avg_grade :
- 5: "Отличная группа".
4: "Хорошая группа".
3: "Удовлетворительная группа".
2, 1: "Слабая группа".
default: "Ошибка вычислений".

Вариант 2. Тема: «Кредитный скоринг в банке»

Контекст: Вы – кредитный инспектор в банке. Вам нужно оценить 4 заявки на потребительский кредит. Решение принимается на основе возраста, дохода, кредитной истории и суммы кредита.

Техническое задание: Введите 4 целых числа: age – возраст (18–70), $income$ – ежемесячный доход (тыс. руб.), $loan_amount$ – сумму кредита (тыс. руб.) и $credits$ – количество действующих кредитов (0–5).

1. Проверьте возраст. Если возраст < 18 или возраст > 70 , выведите "Возраст вне допустимого диапазона" и зафиксируйте отказ.
2. Проверьте доход. Если доход < 30 тыс. руб., выведите "Недостаточный доход" и зафиксируйте отказ.
3. Проверьте сумму кредита. Если сумма $> доход * 12$, выведите "Сумма превышает годовой доход" и зафиксируйте отказ.

4. Подсчитайте общее количество отказов по всем причинам.
5. Используйте оператор switch для определения категории клиента на основе количества кредитов:
0–1: "Надёжный",
2–3: "Средний",
4–5: "Высокий риск".
6. Вычислите модуль разницы между доходом и суммой кредита. Если разница < 10 тыс. руб., выведите "Критический баланс".

Вариант 3. Тема: «Доставка еды из ресторана»

Контекст: Вы – менеджер службы доставки. Нужно рассчитать стоимость доставки заказа на основе расстояния, веса заказа, времени суток и категории клиента.

Техническое задание: Введите 4 целых числа: *distance* – расстояние (км), *weight* – вес заказа (кг), *time* – время суток (1 – день, 2 – вечер, 3 – ночь) и *status* – статус клиента (1 – обычный, 2 – постоянный, 3 – премиум).

1. Вычислите базовую стоимость доставки по формуле:

$$100 \text{ руб.} + 20 \text{ руб.} * \text{расстояние.}$$

2. Проверьте вес. Если вес > 10 кг, добавьте 50 руб. за перевес.

3. Используйте оператор switch для времени суток:

день – без наценки,

вечер – +20%,

ночь – +50%.

Проверьте корректность ввода времени (если не 1–3 – выведите ошибку).

4. Используйте оператор switch для статуса клиента:

обычный – без скидки,

постоянный – 10%,

премиум – 20%.

Проверьте корректность ввода статуса (если не 1–3 – выведите ошибку).

5. Проверьте расстояние и вес. Если расстояние > 10 км и вес > 15 кг, выведите "Крупный заказ, требуется грузовой транспорт".

6. Вычислите модуль разницы между расстоянием и весом. Если модуль > 20, выведите "Нестандартное соотношение параметров".

Вариант 4. Тема: «Кинотеатр: Выбор сеанса»

Контекст: Вы – администратор кинотеатра. Помогаете посетителю выбрать фильм и рассчитать стоимость билета на основе возраста, дня недели, жанра и наличия купона.

Техническое задание: Введите 4 целых числа: *age* – возраст (0–120), *day* – день недели (1–7), *genre* – жанр (1 – боевик, 2 – комедия, 3 – ужасы, 4 – мультфильм) и *coupon* – наличие купона (1 – есть, 0 – нет).

1. Проверьте возраст. Если возраст < 6, разрешите доступ только к мультфильмам (жанр 4).
2. Проверьте жанр и возраст. Если жанр – ужасы и возраст < 16, выведите "Доступ запрещён".
3. Используйте оператор switch для жанра: назначьте цену (боевик – 500 руб., комедия – 350 руб., ужасы – 400 руб., мультфильм – 300 руб.).
4. Используйте оператор switch для дня недели: выходные (6–7) – +20%, будни – без наценки.
5. Проверьте наличие купона и возраст. Если купон есть и возраст > 18, примените скидку 15%.
6. Вычислите модуль разницы между возрастом и номером дня недели. Если модуль < 3, выведите "Счастливый билет".

Вариант 5. Тема: «Автосалон: Подбор автомобиля»

Контекст: Вы – менеджер автосалона. Помогаете клиенту выбрать автомобиль на основе бюджета, типа кузова, мощности двигателя и пробега.

Техническое задание: Введите 4 целых числа: *budget* – бюджет (тыс. руб.), *body_type* – тип кузова (1 – седан, 2 – хэтчбек, 3 – внедорожник, 4 – купе), *power* – мощность (л.с.) и *mileage* – пробег (тыс. км).

1. Проверьте бюджет. Если бюджет < 500 тыс. руб., ограничьте выбор только седанами и хэтчбеками (типы 1 и 2).
2. Проверьте мощность и бюджет. Если мощность > 200 л.с. и бюджет < 1000 тыс. руб., выведите "Недостаточно средств для мощного авто".
3. Используйте оператор switch для типа кузова: назначьте базовую цену:
седан – 800 тыс.,
хэтчбек – 700 тыс.,
внедорожник – 1200 тыс.,
купе – 1500 тыс.
4. Проверьте пробег. Если пробег > 100 тыс. км, уменьшите цену на 20%.
5. Проверьте пробег и мощность. Если пробег < 30 тыс. км и мощность > 150 л.с., выведите "Спортивный автомобиль".

6. Вычислите модуль разницы между бюджетом и базовой ценой. Если разница < 100 тыс. руб., выведите "Идеальное совпадение".

Вариант 6. Тема: «Зоопарк: Классификация животных»

Контекст: Вы – зоолог. Классифицируете 4 животных по весу, росту, продолжительности жизни и типу питания.

Техническое задание: Введите для 4 животных: вес (кг), рост (см), продолжительность жизни (лет), тип питания (1 – травоядное, 2 – хищник, 3 – всеядное). Введите $k1$ – средний вес (500 кг) и $k2$ – средняя продолжительность жизни (20 лет).

1. Для каждого животного определите категорию по размеру:
 - a. Если вес $> k1$ и рост > 150 см, присвойте категорию "Крупное".
 - b. Если вес < 50 кг и рост < 50 см, присвойте категорию "Маленькое".
 - c. В остальных случаях присвойте категорию "Среднее".
2. Подсчитайте количество травоядных, хищников и всеядных.
3. Используйте оператор switch для типа питания: выведите "Травоядное", "Хищник" или "Всеядное".
4. Проверьте наличие животного с продолжительностью жизни 0. Если такое есть, выведите "Ошибка ввода".
5. Проверьте тип питания и продолжительность жизни. Если тип питания – хищник и продолжительность жизни > 30 лет, выведите "Долгожитель-хищник".
6. Проверьте корректность ввода типа питания (1–3). При ошибке выведите сообщение.

Вариант 7. Тема: «Гостиница: Бронирование номеров»

Контекст: Вы – менеджер гостиницы. Рассчитываете стоимость проживания для клиента на основе категории номера, количества ночей, сезона и количества гостей.

Техническое задание: Введите 4 целых числа: *category* – категорию номера (1 – эконом, 2 – стандарт, 3 – люкс, 4 – президентский), *nights* – количество ночей (1–30), *season* – сезон (1 – сезон минимального спроса, 2 – межсезонье, 3 – высокий сезон) и *guests* – количество гостей (1–6).

1. Используйте оператор switch для категории номера: назначьте базовую цену за ночь:

эконом – 2000 руб.,
стандарт – 3500 руб.,
люкс – 6000 руб.,
президентский – 12000 руб.
2. Используйте оператор switch для сезона: назначьте наценку:

сезон минимального спроса – -5%,

межсезонье – без наценки,

высокий сезон – +50%.

3. Проверьте количество гостей. Если гостей > 3 , добавьте 500 руб./ночь за каждого дополнительного гостя.
4. Проверьте количество ночей. Если ночей > 10 , примените скидку 10%.
5. Проверьте категорию и сезон. Если категория – президентский и сезон – высокий, выведите "Требуется предоплата 50%".
6. Вычислите модуль разницы между количеством ночей и количеством гостей. Если модуль < 3 , выведите "Семейный тариф".

Вариант 8. Тема: «Стройка: Расчёт материалов»

Контекст: Вы – прораб на стройке. Рассчитываете количество строительных материалов для 4-х объектов. Нужно определить, какой объект требует срочной доставки.

Техническое задание: Введите для 4 объектов: площадь (м^2), высоту (м), количество этажей и тип здания (1 – жилое, 2 – коммерческое, 3 – промышленное). Введите $k1$ – критическая площадь (1000 м^2) и $k2$ – критическая высота (50 м).

1. Для каждого объекта вычислите объём по формуле: *площадь * высота*.
2. Проверьте объём. Если объём $> 5000 \text{ м}^3$, выведите "Крупный объект".
3. Проверьте площадь и высоту. Если площадь $> k1$ и высота $> k2$, выведите "Срочная доставка материалов".
4. Используйте оператор switch для типа здания: жилое – бетон М300, коммерческое – бетон М400, промышленное – бетон М500.
5. Подсчитайте количество объектов, требующих срочной доставки.
6. Найдите максимальный и минимальный объём объектов. Вычислите модуль разности. Если разность > 10000 , выведите "Разнородные объекты".

Вариант 9. Тема: «Аптека: Расчёт дозировки лекарства»

Контекст: Вы – фармацевт. Рассчитываете дозировку лекарства для 4 пациентов на основе веса, возраста, уровня сахара в крови и типа заболевания.

Техническое задание: Введите для 4 пациентов: вес (кг), возраст (лет), уровень сахара (ммоль/л), тип заболевания (1 – диабет, 2 – гипертония, 3 – инфекция). Введите $k1$ – максимальный вес для стандартной дозы (70 кг) и $k2$ – критический уровень сахара (7 ммоль/л).

1. Используйте оператор switch для типа заболевания: назначьте базовую дозу (диабет – 50 мг, гипертония – 100 мг, инфекция – 200 мг).
2. Проверьте вес. Если вес $> k1$, увеличьте дозу на 20%.
3. Проверьте возраст. Если возраст > 60 , уменьшите дозу на 10%.
4. Проверьте уровень сахара и тип заболевания. Если уровень сахара $> k2$ и тип заболевания – диабет, увеличьте дозу на 30%.

5. Подсчитайте количество пациентов с уровнем сахара выше k_2 .
6. Найдите максимальный и минимальный уровень сахара. Вычислите модуль разности. Если разность > 5 , выведите "Нестабильная группа".

Вариант 10. Тема: «Игровой клуб: Турнирная таблица»

Контекст: Вы – администратор игрового клуба. Анализируете результаты 4 игроков в турнире по трём показателям: количество побед, поражений и ничьих.

Техническое задание: Введите для 4 игроков: победы (0–20), поражения (0–20), ничьи (0–20). Введите k_1 – количество побед для выхода в плей-офф (10) и k_2 – количество поражений для вылета (12).

1. Для каждого игрока вычислите общее количество очков по формуле:

$$\text{победы} * 3 + \text{ничьи}.$$

2. Проверьте победы. Если победы $\geq k_1$, зафиксируйте выход игрока в плей-офф.
3. Проверьте поражения. Если поражения $\geq k_2$, зафиксируйте вылет игрока.
4. Вычислите ранг игрока по формуле: $\text{rank} = \text{очки} / 10$ (целочисленное деление).

Используйте оператор switch для ранга игрока на основе значения rank :

0: "Новичок" (0–9 очков),

1: "Новичок" (10–19 очков),

2: "Любитель" (20–29 очков),

3: "Любитель" (30–39 очков),

4: "Профессионал" (40–49 очков),

5: "Профессионал" (50–59 очков),

6: "Мастер" (60 очков),

default: "Неопределённый ранг".

5. Подсчитайте количество игроков, прошедших в плей-офф.
6. Найдите максимальный и минимальный разрыв между победами и поражениями (победы - поражения) по модулю. Если разрыв > 15 , выведите "Нестабильная игра".

Вариант 11. Тема: «Кафе: Расчёт заказа»

Контекст: Вы – менеджер кафе. Помогаете официанту рассчитать сумму заказа с учётом категории блюда, количества порций, времени посещения и типа клиента.

Техническое задание: Введите 4 целых числа: $category$ – категорию блюда (1 – салат, 2 – суп, 3 – горячее, 4 – десерт), $portions$ – количество порций (1–10), $time$ – время посещения (1 – утро, 2 – день, 3 – вечер) и $client$ – тип клиента (1 – обычный, 2 – постоянный, 3 – бизнес-ланч).

1. Используйте оператор switch для категории блюда, назначьте цену за порцию:

салат – 300 руб.,

суп – 350 руб.,

горячее – 500 руб.,

десерт – 250 руб.

2. Используйте оператор switch для времени: утро – скидка 10%, день – без скидки, вечер – +15%.
3. Используйте оператор switch для типа клиента: обычный – без скидки, постоянный – 10%, бизнес-ланч – 20% (но только в дневное время).
4. Проверьте общую сумму. Если сумма > 5000 руб., выведите "Крупный заказ".
5. Проверьте если тип клиента (НЕ бизнес-ланч) и общая сумма заказа > 3000 руб., выведите "Десерт в подарок".
6. Вычислите модуль разницы между количеством порций и номером времени. Если модуль < 2, выведите "Стандартный заказ".

Вариант 12. Тема: «Аэропорт: Регистрация рейса»

Контекст: Вы – диспетчер аэропорта. Проверяете статус 4 рейсов по времени задержки, типу самолёта, количеству пассажиров и направлению (внутренний/международный).

Техническое задание: Введите для 4 рейсов: задержка (мин, 0–300), тип самолёта (1 – Boeing, 2 – Airbus, 3 – Embraer), количество пассажиров (0–300), направление (1 – внутренний, 2 – международный). Введите k_1 – критическая задержка (60 мин) и k_2 – максимальная вместимость (200 чел.).

1. Проверьте задержку. Если задержка > k_1 , зафиксируйте рейс как задержанный.
2. Проверьте количество пассажиров и тип самолёта. Если пассажиров > k_2 и тип самолёта – Embraer, выведите "Самолёт перегружен".
3. Используйте оператор switch для типа самолёта: выведите название.
4. Используйте оператор switch для направления: внутренний – без доп. проверок, международный – выведите "Требуется паспортный контроль".
5. Подсчитайте количество задержанных рейсов.
6. Найдите максимальную и минимальную задержку. Вычислите модуль разности. Если разность > 120, выведите "Неравномерная задержка рейсов".
7. Проверьте наличие рейса с 0 пассажиров. Если такой есть, выведите "Пустой рейс".

Контрольные вопросы

1. Что такое полная и неполная форма условного оператора if? В каких случаях используется каждая из них? Приведите примеры синтаксиса.
2. Выражения какого типа могут определять условия в условном операторе? Может ли условие быть выражением целого типа и почему?

3. Перечислите операции сравнения (отношения). Каков результат их выполнения и к какому типу данных он относится?
4. Какие логические операции поддерживаются в языке Си? В каком порядке они выполняются (приоритет) и какие значения возвращаются в результате их применения?
5. Когда применяется вложение условных операторов?
6. Как правильно записать вложенные условные операторы? На что следует обращать внимание при форматировании и расстановке фигурных скобок?
7. Что такое оператор выбора switch-case? Как им пользоваться и в каких случаях его применение предпочтительнее каскада вложенных if-else?
8. Какие ограничения накладываются на выражение в операторе switch? Какие типы данных допустимы, а какие запрещены?
9. Для чего используется ключевое слово break в операторе switch-case? Что произойдёт, если опустить break в одной из ветвей case?

Лабораторная работа №4. Циклы с известным числом повторений

Цель работы – изучение операторов организации циклов с известным числом повторений (for) и получение навыков программирования циклических алгоритмов при обработке при обработке многомерных массивов и последовательностей данных.

Методические указания

При разработке программы, использующей циклы с известным числом повторений на языке Си, необходимо использовать:

- **одномерные массивы** для хранения последовательности однотипных данных, доступ к элементам которых осуществляется с помощью одного индекса. Обработка элементов одномерного массива выполняется, как правило, с помощью одного цикла for, последовательно перебирающего индексы массива;
- **двумерные массивы (матрицы)** для хранения данных, представленных в виде таблицы, состоящей из строк и столбцов. Доступ к элементам двумерного массива осуществляется с помощью двух индексов: номера строки и номера столбца;
- оператор цикла **for** для организации повторяющихся вычислений, когда количество итераций известно заранее, с управляющей переменной, которая инициализируется, проверяется и изменяется на каждом шаге цикла;
- вложенные циклы **for** для обработки многомерных массивов, где внешний цикл перебирает строки, а внутренний – столбцы, что позволяет последовательно обрабатывать все элементы таблицы;
- операторы управления потоком выполнения **break** и **continue** для изменения стандартного поведения цикла: **break** обеспечивает досрочный выход из цикла при выполнении определённого условия, а **continue** позволяет пропустить оставшуюся часть

текущей итерации и перейти к следующей, что делает логику обработки данных с исключительными ситуациями более гибкой и читаемой;

– генератор псевдослучайных чисел с функциями **rand()** и **srand()** из библиотеки **stdlib.h** для автоматического заполнения массивов исходными данными, а также функцию **time()** из библиотеки **time.h** для инициализации генератора уникальным начальным значением, что обеспечивает различные результаты при каждом запуске программы.

Лабораторное задание

Разработать программу на языке Си, соответствующую номеру варианта и содержащую реализацию циклических алгоритмов с использованием оператора цикла **for**.

1. Выбрать задание, соответствующее номеру варианта и содержащее исходные значения переменных различных типов.
2. Составить программу, осуществляющую заполнение исходных данных путем генерации значений с помощью датчика псевдослучайных чисел и последующий вывод исходных данных. Ввод и вывод сопроводить комментариями.
3. Выполнить программу на ЭВМ.

Варианты лабораторных заданий

Вариант 1. Тема: «Игровой баланс: Герои и уровни»

Контекст: Вы – разработчик RPG. В игре 8 героев, каждый прошел 5 уровней. Отдельно хранятся бонусы за каждый уровень. Нужно найти героя с максимальным суммарным опытом.

Техническое задание:

1. Двумерный массив *Experience[8][5]* (опыт за уровень) – целые числа от 10 до 100.
2. Одномерный массив *Bonus[5]* (бонусные очки за каждый уровень) – целые числа от 1 до 20.
3. Для каждого героя вычислите сумму опыта с учётом бонусов (к опыту добавляется бонус за соответствующий уровень).
4. Если у героя на каком-то уровне опыт оказался меньше 20, прекратите обработку этого героя (он не прошёл уровень) и перейдите к следующему.
5. Найдите героя с максимальной суммой среди прошедших все уровни.
6. Выведите таблицу результатов.

Вариант 2. Тема: «Финансы: Доходы по кварталам и месяцам»

Контекст: Вы – финансовый аналитик. Есть данные о доходах компании за 4 квартала (по 3 месяца в каждом) и инфляции по месяцам. Нужно рассчитать реальные доходы.

Техническое задание:

1. Двумерный массив *Income[4][3]* (доходы по кварталам и месяцам) – целые числа от 100 до 500.

2. Одномерный массив *Inflation[12]* (инфляция для каждого месяца года) – вещественные числа от 1.0 до 1.5 (используйте целые числа 10-15 и делите на 10).
3. Для каждого квартала найдите общий доход с учётом инфляции (доход делим на инфляцию соответствующего месяца).
4. Если в каком-то месяце доход оказался меньше 50 после учёта инфляции, прекратите обработку этого квартала и выведите предупреждение.
5. Найдите квартал с максимальным реальным доходом.
6. Выведите таблицу по месяцам и итоги.

Вариант 3. Тема: «Склад: Товары по категориям»

Контекст: На складе хранятся товары 6 категорий, в каждой категории по 5 видов товаров. Так же хранятся минимальные остатки для каждой категории. Нужно проверить наличие дефицита.

Техническое задание:

1. Двумерный массив *Stock[6][5]* (количество товаров в категориях) – целые числа от 0 до 100.
2. Одномерный массив *MinStock[6]* (минимальный допустимый остаток для каждой категории) – целые числа от 5 до 20.
3. Для каждой категории подсчитайте общее количество товаров.
4. Если в какой-то категории встретился товар с нулевым остатком, прекратите подсчёт для этой категории (дефицит) и перейдите к следующей.
5. Найдите категорию с максимальным общим количеством товаров среди проблемных (не дефицитные).
6. Выведите таблицу по категориям.

Вариант 4. Тема: «Производство: "Золотые руки мастера"»

Контекст: На заводе 8 цехов, каждый выпускает 4 вида деталей. Существует "идеальный вес" для каждого вида деталей. Нужно найти цех, чья продукция в среднем наиболее близка к идеалу.

Техническое задание:

1. Двумерный массив *Weight[8][4]* (вес деталей) – целые числа от 10 до 100.
2. Одномерный массив *Ideal[4]* (идеальный вес для вида деталей) – целые числа от 20 до 80.
3. Для каждого цеха вычислите:
 - a. Средний вес деталей ($\text{сумма} / 4$).
 - b. Среднее отклонение от идеала: $(|вес1-идеал1| + |вес2-идеал2| + \dots) / 4$.

4. Если вес какой-то детали в цехе точно совпадает с идеальным весом для своего вида, прекратите обработку этого цеха и перейдите к следующему.
5. Найдите цех с минимальным средним отклонением.
6. Формат вывода: Таблица с весами деталей, средним весом и средним отклонением для каждого цеха + итоговый ответ.

Вариант 5. Тема: «Сельское хозяйство: "Урожайный сезон"»

Контекст: Агроном анализирует 10 полей, на каждом выращивают 5 культур. Зафиксирована урожайность (ц/га) и норматив для каждой культуры. Нужно найти поле с максимальным превышением норматива.

Техническое задание:

1. Двумерный массив *Harvest[10][5]* (урожайность) – вещественные числа (целые от 10 до 60, разделить на 10: 1.0–6.0 ц/га).
2. Одномерный массив *Norm[5]* (норматив урожайности) – вещественные числа от 1.5 до 4.0.
3. Для каждого поля вычислите:
 - a. Суммарную урожайность.
 - b. Процент выполнения плана: $(\text{сумма} / \text{сумма_нормативов}) * 100\%$.
4. Если урожайность какой-либо культуры превышает норматив более чем в 2 раза, прекратите обработку этого поля (аномальный урожай) и перейдите к следующему.
5. Найдите поле с максимальным процентом выполнения плана.
6. Формат вывода: Таблица с урожайностью, суммой и процентом для каждого поля + комментарий статуса ("Отлично" > 120%, "Норма" 80–120%, "Плохо" < 80%).

Вариант 6. Тема: «IT-инфраструктура: "Нагрузка на серверы"»

Контекст: Системный администратор отслеживает нагрузку на 6 серверов по 8 параметрам (CPU, RAM, I/O, Network и др.). Есть "эталонная нагрузка" для каждого параметра. Нужно найти сервер с минимальной суммарной перегрузкой.

Техническое задание:

1. Двумерный массив *Load[6][8]* (текущая нагрузка) – целые числа от 0 до 100 (проценты).
2. Одномерный массив *Eta[8]* (эталонная нагрузка) – целые числа от 30 до 70.
3. Для каждого сервера вычислите:
 - a. Суммарную перегрузку: сумма (нагрузка - эталон) только для положительных отклонений.
 - b. Количество параметров с превышением эталона.

4. Если хотя бы один параметр имеет нагрузку 100% (полная загрузка), прекратите обработку этого сервера (авария) и перейдите к следующему.
5. Найдите сервер с минимальной суммарной перегрузкой.
6. Формат вывода: Таблица с параметрами, перегрузкой по каждому, итоговой суммой и статусом ("ОК", "Внимание", "Авария").

Вариант 7. Тема: «Медицина: "Группа риска"»

Контекст: Врач анализирует 9 пациентов по 4 показателям здоровья. Есть "норма" и "критический порог" для каждого показателя. Нужно найти пациента с наибольшим количеством критических отклонений.

Техническое задание:

1. Двумерный массив *Health[9][4]* (показатели) – вещественные числа (целые от 10 до 200, /10: 1.0–20.0).
2. Два одномерных массива *Norm[4]* (норма) и *Critical[4]* (критический порог) – вещественные числа от 3.0 до 15.0.
3. Для каждого пациента вычислите:
 - a. Количество критических отклонений (показатель > критический порог).
 - b. Количество незначительных отклонений (показатель > норма1.2 ИЛИ < норма0.8, но не критический).
4. Если все 4 показателя находятся в норме, прекратите обработку этого пациента (здоров) и перейдите к следующему.
5. Найдите пациента с максимальным количеством критических отклонений.
6. Формат вывода: Таблица с показателями, нормой, критическим порогом, количеством отклонений и статусом ("Здоров", "Группа риска", "Критично").

Вариант 8. Тема: «Строительство: "Сметный лимит"»

Контекст: Прораб контролирует 8 объектов по 5 статьям расходов. Есть "лимит" и "штрафной порог" для каждой статьи. Нужно найти объект с максимальным количеством штрафных превышений.

Техническое задание:

1. Двумерный массив *Cost[8][5]* (фактические расходы) – целые числа от 10 до 100 (тыс. руб.).
2. Два одномерных массива *Limit[5]* (лимит) и *Penalty[5]* (штрафной порог) – целые числа от 20 до 70.
3. Для каждого объекта вычислите:
 - a. Количество штрафных превышений (расход > штрафной порог).
 - b. Суммарное превышение лимита: сумма (расход - лимит) для положительных значений.

4. Если все 5 статей имеют превышение лимита, прекратите обработку этого объекта (полный перерасход) и перейдите к следующему.
5. Найдите объект с максимальным количеством штрафных превышений.
6. Формат вывода: Таблица с расходами, лимитами, штрафными порогами, количеством штрафов и статусом ("Норма", "Перерасход", "Штраф").

Вариант 9. Тема: «Экология: "Красная книга"»

Контекст: Эколог исследует 5 заповедников по 7 видам животных. Есть "минимальная популяция" и "критическая черта" для каждого вида. Нужно найти заповедник с наибольшим количеством исчезающих видов.

Техническое задание:

1. Двумерный массив *Population[5][7]* (численность популяции) – целые числа от 0 до 100.
2. Два одномерных массива *MinPop[7]* (минимальная популяция) и *CritPop[7]* (критическая черта) – целые числа от 5 до 40.
3. Для каждого заповедника вычислите:
 - a. Количество исчезающих видов (популяция < критическая черта).
 - b. Количество видов ниже минимальной популяции.
4. Если в заповеднике есть вид с нулевой популяцией, прекратите обработку этого заповедника (вымирание) и перейдите к следующему.
5. Найдите заповедник с максимальным количеством исчезающих видов.
6. Формат вывода: Таблица с популяциями, критической чертой, минимальной популяцией, количеством исчезающих и статусом ("Благополучно", "Тревога", "Катастрофа").

Вариант 10. Тема: «Телекоммуникации: "Качество сигнала"»

Контекст: Инженер проверяет 10 базовых станций по 4 показателям. При обнаружении станции с низким качеством всех сигналов проверка прекращается.

Техническое задание:

1. Двумерный массив *Signal[10][4]* (качество сигнала) – целые числа от 0 до 100.
2. Два одномерных массива *Min[4]* (минимальный порог) и *Optimal[4]* (оптимальный уровень) – целые числа от 30 до 80.
3. Для каждой станции подсчитайте количество значений ниже минимального порога.
4. Если у станции все 4 значения ниже минимального порога, прекратите выполнение программы и выведите "Станция X требует технического обслуживания".
5. Если остановки не произошло, найдите станцию с максимальным количеством превышений оптимального уровня.

6. Формат вывода: Таблица с результатами по станциям до остановки.

Вариант 11. Тема: «Логистика: "Соблюдение сроков"»

Контекст: Логист анализирует 15 маршрутов по 3 этапам доставки. При обнаружении маршрута со значительными задержками на всех этапах анализ прекращается.

Техническое задание:

1. Двумерный массив *Time[15][3]* (фактическое время) – целые числа от 10 до 200.
2. Одномерный массив *Ideal[3]* (идеальное время) – целые числа от 20 до 100.
3. Для каждого маршрута подсчитайте количество этапов с опозданием.
4. Если на маршруте все 3 этапа имеют опоздание более чем на 30 минут, прекратите выполнение программы и выведите "Маршрут X требует оптимизации".
5. Если остановки не произошло, найдите маршрут с минимальным суммарным опозданием.
6. Формат вывода: Таблица с результатами по маршрутам до остановки.

Вариант 12. Тема: «Сельское хозяйство: "Рекордные урожаи"»

Контекст: Агроном анализирует 7 полей по 5 культурам. При обнаружении поля с выдающейся урожайностью всех культур анализ прекращается.

Техническое задание:

1. Двумерный массив *Harvest[7][5]* (урожайность) – вещественные числа (целые от 10 до 80, /10: 1.0–8.0 т/га).
2. Два одномерных массива *Record[5]* (рекордный урожай) и *Average[5]* (среднегодовой урожай) – вещественные числа от 2.0 до 6.0.
3. Для каждого поля подсчитайте количество культур, побивших рекорд.
4. Если на поле все 5 культур побили рекорд, прекратите выполнение программы и выведите "Поле X показывает рекордную урожайность".
5. Если остановки не произошло, найдите поле с максимальным количеством культур, побивших рекорд.
6. Формат вывода: Таблица с результатами по полям до остановки.

Контрольные вопросы

1. Что такое массив? Чем одномерный массив отличается от двумерного (матрицы)? Как элементы массива располагаются в памяти компьютера?
2. Что такое индекс элемента массива? С какого значения начинается индексация в языке Си?
3. Что такое цикл? Какие виды циклов существуют в языке Си и в чём их принципиальное различие?

4. Каков порядок действий при выполнении цикла `for`? Опишите последовательность выполнения всех трёх выражений в заголовке цикла.
5. Как организовать арифметический цикл с помощью оператора `for`? Приведите пример цикла, выполняющего суммирование чисел от 1 до N.
6. Что такое вложенный цикл? В каких случаях применяются вложенные циклы и как они организуются в языке Си?
7. Сколько раз в общей сложности выполняется тело вложенного цикла? Как рассчитать общее количество итераций для внешнего и внутреннего циклов?
8. Каким образом организуется обработка многомерных массивов с использованием вложенных циклов? Какой цикл (внешний или внутренний) отвечает за перебор строк, а какой – за перебор столбцов?
9. Как и когда используются операторы `break` и `continue` в циклах? В чём разница между ними и как они изменяют стандартное поведение цикла?

Лабораторная работа №5. Циклы с неизвестным числом повторений

Цель работы – изучение операторов организации циклов с неизвестным числом повторений (`while`, `do-while`) и получение навыков программирования циклических алгоритмов при обработке данных, когда количество итераций заранее не определено и зависит от выполнения некоторого условия.

Методические указания

При разработке программы, использующей циклы с известным числом повторений на языке Си, необходимо использовать:

- оператор цикла **`while`** для организации повторяющихся вычислений, когда количество итераций заранее неизвестно и зависит от выполнения условия, которое проверяется до начала выполнения тела цикла (если условие изначально ложно, тело цикла не выполняется ни разу);
- оператор цикла **`do-while`** для организации повторяющихся вычислений, когда количество итераций заранее неизвестно, но тело цикла должно быть выполнено как минимум один раз, поскольку условие проверяется после выполнения тела цикла;
- операторы управления потоком выполнения **`break`** и **`continue`** для изменения стандартного поведения цикла: **`break`** обеспечивает досрочный выход из цикла при выполнении определённого условия, а **`continue`** позволяет пропустить оставшуюся часть текущей итерации и перейти к следующей, что делает логику обработки данных с исключительными ситуациями более гибкой и читаемой;
- генератор псевдослучайных чисел с функциями **`rand()`** и **`srand()`** из библиотеки **`stdlib.h`** для автоматического заполнения массивов исходными данными, а также функцию **`time()`** из библиотеки **`time.h`** для инициализации генератора уникальным начальным значением, что обеспечивает различные результаты при каждом запуске программы.

Лабораторное задание

Разработать программу на языке Си, соответствующую номеру варианта и содержащую реализацию циклических алгоритмов с использованием операторов цикла с неизвестным числом повторений (while или do-while).

1. Выбрать задание, соответствующее номеру варианта и содержащее исходные значения переменных различных типов.
2. Составить программу, осуществляющую заполнение исходных данных путем генерации значений с помощью датчика псевдослучайных чисел и последующий вывод исходных данных. Ввод и вывод сопроводить комментариями.
3. Выполнить программу на ЭВМ.

Варианты лабораторных заданий

Вариант 1. Тема: «Копилка для путешествия»

Контекст: Вы планируете путешествие и каждый месяц откладываете деньги в специальную копилку. Банк ежемесячно начисляет небольшой процент на текущую сумму, поэтому размер накоплений постепенно меняется.

Техническое задание: Необходимо определить, через сколько месяцев сумма накоплений достигнет стоимости путешествия.

1. Входные данные: начальная сумма S , ежемесячное пополнение P , годовая процентная ставка r и стоимость путешествия N .
2. В начале каждого месяца к накопленной сумме добавляется P .
3. Затем начисляются проценты: $S = S + S * r / 1200$.
4. Если начисленные проценты за месяц составили менее 1 рубля, этот месяц не учитывается при подсчёте месяцев с существенным начислением.
5. Необходимо вычислить количество месяцев, итоговую сумму и количество месяцев, в которых начисление составило не менее 1 рубля.
6. Если после очередного месяца сумма достигла или превысила N , накопление прекращается.
7. Расчёт продолжается до тех пор, пока стоимость путешествия не будет накоплена либо пока не будет достигнут максимальный срок накопления в 120 месяцев.
8. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 2. Тема: «Спасение принцессы»

Контекст: Герой пробирается через подземелье, чтобы найти принцессу. Дорога состоит из неизвестного количества комнат: в каждой комнате героя ждёт случайное событие, которое может помочь ему или отнять часть здоровья.

Техническое задание: Начальное здоровье героя – 100, количество золота – 0. При входе в каждую комнату случайным образом определяется событие:

1. найдено золото – от 10 до 30 монет;
2. ловушка – герой теряет от 5 до 20 единиц здоровья;
3. зелье – герой восстанавливает от 5 до 15 единиц здоровья, но здоровье не может превышать 100;
4. пустая комната – ничего не происходит.

Используйте генерацию случайных чисел для выбора события и его параметров.

1. Если герой нашёл золото, оно добавляется к общему запасу.
2. Если герой попал в ловушку, вывести сообщение о потере здоровья.
3. Если после события здоровье героя стало меньше или равно 30, вывести предупреждение.
4. После трёх комнат подряд без потери здоровья герой получает бонус 20 золотых.
5. Для спасения принцессы необходимо собрать не менее 150 золотых.
6. Если здоровье героя становится меньше или равно нулю, приключение заканчивается поражением.
7. Если герой посетил 30 комнат, не собрав необходимое количество золота, приключение заканчивается: путь оказался слишком длинным.

Необходимо определить количество посещённых комнат, итоговое здоровье, количество золота и результат приключения.

Вариант 3. Тема: «Разгон двигателя»

Контекст: Инженер проверяет, как быстро двигатель выходит на рабочую температуру. После каждого измерения система рассчитывает новое значение температуры, которое постепенно приближается к заданному рабочему значению.

Техническое задание: Температура двигателя на каждом шаге рассчитывается по формуле:

$$T_{new} = T + k * (T_{target} - T),$$

где T – текущая температура, T_{target} – рабочая температура, k – коэффициент нагрева.

1. Входные данные: начальная температура T_0 , рабочая температура T_{target} , коэффициент k (от 0.1 до 0.9) и требуемая точность eps .
2. После каждого шага выводить новую температуру.
3. Определить первый момент, когда изменение температуры между двумя последовательными измерениями станет меньше заданной точности.
4. Если количество измерений достигнет 100, наблюдение необходимо завершить.
5. Расчёт продолжается до достижения требуемой точности, перегрева двигателя или ограничения по количеству измерений.
6. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 4. Тема: «Фильтр для очистки воды»

Контекст: На станции очистки воды контролируется концентрация загрязняющего вещества. После каждого этапа очистки концентрация уменьшается, но эффективность фильтра зависит от текущего состояния воды.

Техническое задание: После каждого этапа концентрация рассчитывается по формуле:

$$C_{new} = C - C * k,$$

где C – текущая концентрация, а k – коэффициент очистки.

1. Входные данные: начальная концентрация C , коэффициент очистки k и допустимая концентрация N .
2. После каждого этапа выводить текущую концентрацию.
3. Если уменьшение концентрации за очередной этап оказалось меньше 0.01, считать фильтр практически неэффективным.
4. Если уменьшение концентрации оказалось менее 1% от предыдущего значения, вывести предупреждение о низкой эффективности.
5. Определить количество этапов, необходимое для достижения нормы.
6. Очистка продолжается до достижения допустимой концентрации, потери эффективности фильтра или обнаружения некорректного значения.
7. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 5. Тема: «Тренировка перед марафоном»

Контекст: Спортсмен готовится к марафону и после каждой тренировки фиксирует пройденную дистанцию. Некоторые измерения могут оказаться ошибочными, а достижение марафонской дистанции означает завершение подготовки.

Техническое задание: Пользователь последовательно вводит результаты тренировок.

1. Значение 0 означает окончание ввода.
2. Отрицательная дистанция считается ошибочной.
3. Ошибочные значения не учитываются в статистике и не прерывают последовательность корректных тренировок.
4. Определить максимальную и среднюю дистанцию среди корректных тренировок.
5. Определить максимальное количество последовательных тренировок, на которых результат не уменьшался по сравнению с предыдущей корректной тренировкой.
6. Если спортсмен преодолел дистанцию 42.195 км, подготовка считается завершённой досрочно.
7. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 6. Тема: «Бой с драконом»

Контекст: Герой сражается с драконом. Сила каждой атаки определяется случайным образом, поэтому заранее неизвестно, сколько ударов потребуется для победы.

Техническое задание:

1. Начальное здоровье героя – 100, дракона – 150.
2. Сила атаки героя случайно определяется в диапазоне от 10 до 30.
3. Сила атаки дракона – от 5 до 25.
4. Использовать `srand()` и `rand()` для моделирования случайных атак.
5. Если атака героя нанесла менее 15 единиц урона, она считается слабой, и дракон не наносит ответный удар.
6. Если здоровье героя становится меньше или равно 30, необходимо вывести предупреждение о критическом состоянии.
7. Подсчитать количество атак героя и дракона.
8. Бой продолжается до тех пор, пока здоровье одного из участников не станет равным нулю или отрицательным.
9. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 7. Тема: «Контроль качества на заводе»

Контекст: На производственной линии последовательно проверяются детали. Датчик измеряет массу каждой детали, однако отдельные измерения могут оказаться некорректными.

Техническое задание: Пользователь последовательно вводит массу деталей в граммах.

1. Значение 0 завершает ввод.
2. Отрицательное значение считается ошибочным и не должно учитываться.
3. Определить минимальную и максимальную массу корректных деталей.
4. Вычислить среднюю массу.
5. Подсчитать количество деталей, масса которых находится в диапазоне от 95 до 105 граммов.
6. Если обнаружено три детали тяжелее 110 граммов, производство необходимо немедленно остановить.
7. Ввод продолжается до специального значения, аварийной остановки или окончания партии.
8. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 8. Тема: «Поездка на электромобиле»

Контекст: Электромобиль отправляется в поездку с полностью заряженной батареей. Маршрут состоит из участков различной длины, поэтому расход энергии на каждом шаге меняет остаток заряда.

Техническое задание:

1. Входные данные: ёмкость батареи B , средний расход q кВт·ч на 100 км и длины участков маршрута.
2. Длина очередного участка вводится пользователем.
3. Значение 0 означает окончание маршрута.
4. Отрицательная длина участка считается ошибочной и не учитывается.
5. Перед прохождением участка определить необходимое количество энергии.
6. Если имеющегося заряда недостаточно для прохождения участка, поездка завершается.
7. Если заряд после прохождения участка становится меньше 10% ёмкости батареи, вывести предупреждение.
8. Определить общую пройденную дистанцию и остаток энергии.
9. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 9. Тема: «Восстановление после тренировки»

Контекст: После интенсивной тренировки спортсмен отслеживает частоту пульса. Пульс постепенно возвращается к нормальному значению, но скорость восстановления зависит от текущего состояния организма.

Техническое задание: После каждого измерения пульс рассчитывается по формуле:

$$P_{new} = P - k * (P - P_{normal}),$$

1. Входные данные: начальный пульс P , нормальный пульс P_{normal} , коэффициент восстановления k (от 0.1 до 0.9) и точность eps .
2. После каждого шага выводить значение пульса.
3. Определить первый момент, когда изменение между двумя последовательными измерениями станет меньше заданной точности.
4. Если количество измерений превысит 50, наблюдение прекращается.
5. Наблюдение продолжается до стабилизации показателя, появления аномального значения или достижения ограничения по количеству измерений.
6. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 10. Тема: «Эксперимент с растением»

Контекст: Биолог наблюдает за ростом растения. В начале растение увеличивается быстрее, но после достижения определённой высоты его рост замедляется.

Техническое задание:

1. Входные данные: начальная высота H , процент роста p и требуемая высота N .
2. Каждый день вычислять новую высоту по формуле: $H = H + H * p / 100$.
3. После достижения 80% от требуемой высоты коэффициент роста уменьшается в два раза.
4. Определить количество дней, необходимое для достижения целевой высоты.
5. Если прирост за день становится меньше 0.01 см, считать рост практически остановившимся.
6. Если рост прекратился до достижения целевой высоты, сообщить, что растение не достигло требуемой высоты.
7. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 11. Тема: «Энергия космического корабля»

Контекст: Космический корабль летит к станции и расходует энергию на каждом этапе полёта. Одновременно солнечные панели восполняют часть энергии, причём количество восстановленной энергии меняется случайным образом.

Техническое задание:

1. Входные данные: начальный запас энергии E , расход энергии за этап C и необходимое количество этапов N .
2. На каждом этапе сначала расходуется C единиц энергии.
3. Затем солнечные панели восстанавливают случайное количество энергии от 5 до 15 единиц. Использовать `srand()` и `rand()`.
4. Если запас энергии после расхода становится меньше 20 единиц, вывести предупреждение.
5. Если энергии недостаточно для продолжения полёта, экспедиция прекращается.
6. Если солнечные панели восстановили 15 единиц энергии, этап считается успешным.
7. Определить количество пройденных этапов, количество успешных этапов и остаток энергии.
8. Полёт продолжается до прохождения необходимого количества этапов или невозможности продолжать движение.
9. Формат вывода: Таблица с результатами по шагам до остановки.

Вариант 12. Тема: «Поиск кода доступа»

Контекст: Электронный сейф защищён числовым кодом. Система случайным образом выбирает секретное число, а пользователь пытается его угадать.

Техническое задание:

1. Сгенерировать секретное число от 1 до 100.
2. Использовать `srand()` и `rand()`.
3. Пользователь вводит варианты ответа.
4. После каждой попытки сообщать, больше или меньше введённое число относительно секрета.
5. Если введено число вне диапазона от 1 до 100, попытка не засчитывается.
6. После семи засчитанных попыток сейф блокируется.
7. При правильном ответе вывести количество использованных попыток.
8. Формат вывода: Таблица с результатами по шагам до остановки.

Контрольные вопросы

1. Что такое цикл с неизвестным числом повторений? Чем он отличается от цикла с известным числом повторений?
2. Какие циклы с неизвестным числом повторений существуют в языке Си? В чём их принципиальное различие?
3. Как работает цикл `while`? В какой момент проверяется условие продолжения цикла?
4. Как работает цикл `do...while`? Чем порядок его выполнения отличается от `while`?
5. Что произойдёт, если условие цикла `while` / `do...while` изначально ложно? Сколько раз выполнится тело цикла?
6. Чем отличается оператор `while` от оператора `if`?
7. Какие ошибки могут привести к бесконечному циклу? Как определить причину и исправить такую ошибку?

Лабораторная работа №6. Указатели

Цель работы – изучение механизма указателей в языке Си и получение навыков работы с массивами через указатели.

Методические указания

При разработке программы, использующей указатели для работы с массивами на языке Си, необходимо использовать:

- **определение указателей** с использованием оператора `*` (например, `int *ptr;`), которые хранят адреса ячеек памяти и позволяют обращаться к данным по этим адресам;
- **оператор взятия адреса** `&` для получения адреса переменной или элемента массива (например, `&array[0]` или `&variable`);
- **оператор разыменования** `*` для доступа к значению, хранящемуся по адресу, на который указывает указатель (например, `*ptr = 10;`);
- **связь между массивами и указателями:** имя массива является константным указателем на его первый элемент (например, агау эквивалентно `&array[0]`), поэтому доступ к элементам массива может осуществляться как через индексы `array[i]`, так и через указатели `*(array + i)`;
- **адресную арифметику** для навигации по массиву: прибавление целого числа к указателю смещает его на соответствующее количество элементов с учётом размера типа данных (например, `ptr + 1` указывает на следующий элемент массива);
- **указатели на двумерные массивы**, где строка рассматривается как одномерный массив, а доступ к элементам осуществляется через указатели на строки: `*(*(matrix + i) + j)` или `((*(matrix + i))[j])` для доступа к элементу `matrix[i][j]`;
- **массивы указателей** (например, `int *ptrArray[N]`) для хранения адресов различных объектов или строк;
- **генератор псевдослучайных чисел** с функциями `rand()` и `srand()` из библиотеки `stdlib.h` для автоматического заполнения массивов исходными данными, а также функцию `time()` из библиотеки `time.h` для инициализации генератора уникальным начальным значением.

Лабораторное задание

Разработать программу на языке Си, соответствующую **номеру варианта из лабораторной работы № 4** (Циклы с известным числом повторений), в которой все обращения к элементам массивов производятся с использованием указателей, а не индексов.

1. Выбрать задание, соответствующее номеру варианта из лабораторной работы № 4.
2. Составить программу, осуществляющую заполнение исходных данных путем генерации значений с помощью датчика псевдослучайных чисел и последующий вывод исходных данных. **Все обращения к элементам массивов производить с помощью указателей, используя адресную арифметику вместо оператора индексации []**. Ввод и вывод сопроводить комментариями.
1. Выполнить программу на ЭВМ.

Контрольные вопросы

1. Что такое указатель? Что хранится в переменной-указателе и как его инициализировать?
2. Что означают операции получения адреса `&` и разыменования `*`? Как с их помощью работать с данными?

3. Какие операции допустимы при работе с указателями? Как работают инкремент и декремент указателя?
4. Как связаны имя массива и указатель на его первый элемент?
5. Как обратиться к элементу одномерного массива с помощью указателя? Как вычисляется адрес нужного элемента?
6. Как элементы двумерного массива (матрицы) располагаются в памяти? Как вычисляется адрес элемента матрицы по его индексам?
7. Как обратиться к элементу матрицы через указатель? Чем отличается указатель на элемент матрицы от указателя на строку?
8. Какие ошибки могут возникнуть при работе с указателями и массивами? Что произойдёт при выходе за границы массива?

Лабораторная работа №7. Строки

Цель работы – изучение способов представления и обработки строк в языке Си, получение навыков работы с Си-строками, а также освоение стандартных функций библиотек `string.h` и `ctype.h` для ввода, поиска, сравнения, копирования, объединения, разбиения и преобразования строк.

Методические указания

При разработке программы для обработки строк на языке Си необходимо использовать:

- Си-строки – последовательности символов, завершающиеся нуль-терминатором `'\0'`. Нуль-терминатор определяет границу строки и позволяет стандартным функциям языка определить её конец;
- для хранения строк – массивы символов `char st[N]`, где `N` – размер массива;
- функции ввода строк с клавиатуры, например `fgets()`, позволяющие вводить строку целиком, включая пробелы. При использовании `fgets()` необходимо учитывать возможность сохранения символа перевода строки `'\n'`;
- основные функции библиотек `string.h` и `ctype.h`.

Основные функции библиотеки `string.h` представлены в таблице 2.

Таблица 2 – Основные функции библиотеки `string.h`

Функция	Назначение	Пример использования
<code>strlen()</code>	Определение длины строки без учёта нуль-терминатора <code>'\0'</code>	<code>n = strlen(st);</code>
<code>strcpy()</code>	Копирование содержимого одной строки в другую	<code>strcpy(copy, st);</code>
<code>strcat()</code>	Конкатенация (объединение) двух строк	<code>strcat(st, suffix);</code>
<code>strcmp()</code>	Сравнение двух строк. Возвращает 0, если строки равны	<code>if (strcmp(st1, st2) == 0)</code>

strchr()	Поиск первого вхождения заданного символа в строку	p = strchr(st, '@');
strstr()	Поиск первого вхождения подстроки в строку	p = strstr(st, "error");
strtok()	Разбиение строки на отдельные части (токены), например слова	word = strtok(st, " ");

Основные функции библиотеки **ctype.h** представлены в таблице 3.

Таблица 3 – Основные функции библиотеки ctype.h

Функция	Назначение	Пример использования
isalpha()	Проверка, является ли символ буквой	if (isalpha(st[i]))
isdigit()	Проверка, является ли символ цифрой	if (isdigit(st[i]))
isspace()	Проверка, является ли символ пробельным	if (isspace(st[i]))
tolower()	Преобразование буквы в нижний регистр	st[i] = tolower(st[i]);
toupper()	Преобразование буквы в верхний регистр	st[i] = toupper(st[i]);

Лабораторное задание

Разработать программу на языке Си, соответствующую номеру варианта и содержащую реализацию алгоритма обработки строк с использованием стандартных функций библиотек string.h и ctype.h.

1. Выбрать задание, соответствующее номеру варианта и содержащее исходные строки, символы или подстроки, необходимые для выполнения обработки.
2. Составить программу, осуществляющую ввод исходных строк и последующую их обработку в соответствии с условием варианта. Предусмотреть использование не менее трёх стандартных функций для работы со строками. Ввод и вывод сопроводить комментариями.
3. Вывести исходные данные и результат их обработки. Если условием варианта предусмотрена проверка или анализ строки, вывести также соответствующие результаты.
4. Выполнить программу на ЭВМ.

Вариант 1. Тема: «Команда для робота»

Контекст: Домашний робот получает команды пользователя в виде строки. Команда может содержать действие и несколько слов, описывающих объект, например *move to kitchen* или *open front door*.

Техническое задание:

1. Ввод: Одна строка – команда робота.
2. Действие: Разбить команду на слова. Первое слово считать действием, остальные – параметрами. Проверить, является ли действие допустимым: *move*, *open*, *close*, *stop*. Для

команд *move*, *open* и *close* проверить наличие параметров. Объединить все параметры в одну строку и сформировать сообщение о выполнении команды.

3. Функции: `strtok()`, `strcmp()`, `strlen()`, `strcpy()/strcat()`.

4. Вывод: Исходную команду, действие, параметры, длину параметров и итоговое сообщение. При некорректной команде вывести причину ошибки.

Вариант 2. Тема: «Регистрация пользователя»

Контекст: При регистрации в системе пользователь выбирает имя. Система должна проверить его допустимость и привести его к единому формату.

Техническое задание:

1. Вход: Одна строка – имя пользователя.

2. Действие: Проверить длину имени – от 5 до 15 символов. Проверить, что оно содержит только буквы и цифры и хотя бы одну цифру. Проверить, что имя не совпадает с одним из зарезервированных имён: *admin*, *root*, *user*. Привести буквы к нижнему регистру.

3. Функции: `strlen()`, `isalpha()`, `isdigit()`, `tolower()`, `strcmp()`, `strcpy()`.

4. Вывод: Исходное имя, нормализованное имя, результаты отдельных проверок и итоговый статус.

Вариант 3. Тема: «Почтовый фильтр»

Контекст: Почтовый сервер автоматически очищает входящие сообщения от нежелательных слов. Список стоп-слов хранится в отдельной строке.

Техническое задание:

1. Вход: Две строки – текст письма и список стоп-слов через запятую.

2. Действие: Разбить письмо на слова. Разбить список стоп-слов на отдельные элементы. Для каждого слова письма проверить наличие совпадения со стоп-словами без учёта регистра. Если слово найдено в списке, заменить его на [SPAM]. Сформировать новое письмо.

3. Функции: `strtok()`, `strcmp()`, `tolower()`, `strcpy()/strcat()`, `strlen()`.

4. Вывод: Исходное письмо, список стоп-слов, очищенное письмо, количество замен и длину результата.

Вариант 4. Тема: «Умный поиск товаров»

Контекст: Интернет-магазин получает поисковый запрос из нескольких слов. Для упрощённого поиска необходимо найти товары, содержащие ключевые слова запроса.

Техническое задание:

1. Вход: Две строки – поисковый запрос и строка с названиями товаров.

2. Действие: Разбить запрос на слова и названия товаров на отдельные элементы. Для каждого товара проверить, содержит ли он хотя бы одно слово из запроса без учёта

регистра. Подходящие товары добавить в результирующую строку. Один товар должен попасть в результат только один раз.

3. Функции: `strtok()`, `strstr()`, `strcmp()`, `tolower()`, `strcpy()/strcat()`, `strlen()`.
4. Вывод: Исходный запрос, список товаров, найденные товары и их количество.

Вариант 5. Тема: «Телефонная книга»

Контекст: В телефонной книге номера разных людей могут быть записаны в разных форматах. Кроме того, пользователю необходимо найти контакт по имени.

Техническое задание:

1. Вход: Две строки – имя контакта и его номер телефона.
2. Действие: Нормализовать номер: удалить пробелы, скобки и дефисы. Проверить, что после обработки он содержит только цифры и допустимый + в начале. Привести имя контакта к формату с первой заглавной буквой. Сформировать единую запись вида Имя: номер.
3. Функции: `strlen()`, `strcpy()/strcat()`, `isdigit()`, `isspace()`, `toupper()/tolower()`, `strchr()`.
4. Вывод: Исходные данные, нормализованный номер, имя и итоговую запись контакта.

Вариант 6. Тема: «Парсер настроек приложения»

Контекст: Приложение получает настройки одной строкой в формате `ключ=значение`. Перед запуском необходимо проверить несколько параметров.

Техническое задание:

1. Вход: Одна строка – набор параметров вида `key=value`, разделённых пробелами.
2. Действие: Разобрать все параметры. Найти параметры `mode`, `user` и `port`. Проверить, что `mode` имеет значение `debug` или `release`, а `port` состоит только из цифр. Сформировать нормализованную строку конфигурации в формате `mode=...; user=...; port=...`
3. Функции: `strtok()`, `strchr()`, `strcmp()`, `strcpy()/strcat()`, `isdigit()`, `strlen()`.
4. Вывод: Исходную конфигурацию, найденные значения параметров, нормализованную конфигурацию и результаты проверок.

Вариант 7. Тема: «Обработка JSON-ответа сервера»

Контекст: Мобильное приложение получает от сервера ответ с информацией о пользователе в упрощённом JSON-формате. Необходимо извлечь из ответа несколько параметров и проверить их значения перед отображением данных пользователю.

Техническое задание:

1. Вход: Одна строка – JSON-подобный объект фиксированного формата:

```
{"name": "Ivan", "status": "active", "age": "20", "role": "student"}
```

2. Действие: Найти в строке значения параметров *name*, *status*, *age* и *role*. Для каждого параметра определить значение, находящееся после соответствующего ключа. Проверить, что *status* имеет значение *active* или *blocked*, а *age* содержит только цифры. Сформировать итоговое сообщение о пользователе.
3. Функции: `strstr()`, `strchr()`, `strcmp()`, `strlen()`, `strcpy()`, `strcat()`, `isdigit()`.
4. Вывод: Исходный JSON, извлечённые значения параметров, результаты проверок и сформированное сообщение.

Вариант 8. Тема: «Обработка данных CSV»

Контекст: Система интернет-магазина получает информацию о товарах в текстовом формате CSV. Каждая строка содержит название товара, категорию и цену.

Техническое задание:

1. Вход: Одна строка с несколькими записями, разделёнными. Поля одной записи разделены запятыми:

Laptop,Electronics,1200;Phone,Electronics,800;Chair,Furniture,150

2. Действие: Разбить строку на записи товаров, а каждую запись – на отдельные поля. Найти товары заданной категории. Для найденных товаров сформировать строку с их названиями. Дополнительно определить самый длинный товар и количество найденных товаров.
3. Функции: `strtok()`, `strcmp()`, `strchr()`, `strlen()`, `strcpy()`, `strcat()`.
4. Вывод: Исходные данные, найденные товары, их количество и название самого длинного товара.

Вариант 9. Тема: «Секретное сообщение агента»

Контекст: Агент передаёт сообщение, зашифрованное простым шифром Цезаря. Для дополнительной проверки после расшифровки необходимо найти в сообщении кодовое слово.

Техническое задание:

1. Вход: Три значения – зашифрованное сообщение, ключевое слово и величина сдвига.
2. Действие: Расшифровать сообщение с помощью шифра Цезаря. При обработке изменять только буквы, сохраняя пробелы и знаки препинания. После расшифровки найти кодовое слово без учёта регистра. Определить длину сообщения и количество слов.
3. Функции: `strlen()`, `strcpy()`, `strstr()`, `strtok()`, `isalpha()`, `tolower()`.
4. Вывод: Зашифрованное сообщение, расшифрованное сообщение, количество слов, результат поиска кодового слова.

Примечание: Шифр Цезаря – это способ шифрования, при котором каждая буква сообщения заменяется на другую букву, находящуюся на заданное число позиций дальше в алфавите. Это число называется сдвигом. Например, при сдвиге 3: А → Г, Б → Д, В → Е и т. д. После Я алфавит начинается снова с А.

Для расшифровки используется обратный сдвиг: из позиции каждой буквы вычитается значение ключа. Например, если сообщение было зашифровано со сдвигом 3, то $\Gamma \rightarrow \text{A}$, $\text{Д} \rightarrow \text{Б}$, $\text{Е} \rightarrow \text{В}$.

При обработке необходимо учитывать переход через границу алфавита и изменять только буквы сообщения. Пробелы, цифры и знаки препинания должны оставаться без изменений.

Вариант 10. Тема: «Фильтр секретной переписки»

Контекст: Перед отправкой сообщения система должна заменить некоторые служебные слова на специальную метку, а затем зашифровать полученный текст простым шифром Цезаря.

Техническое задание:

1. Ввод: Три значения – текст сообщения, одно запрещённое слово и величина сдвига.
2. Действие: Найти в сообщении все слова, совпадающие с заданным запрещённым словом без учёта регистра, и заменить их на *[SECRET]*. После этого зашифровать полученное сообщение шифром Цезаря, изменяя только буквы. Пробелы и знаки препинания оставить без изменений.
3. Функции: `strtok()`, `strcmp()`, `tolower()`, `strcpy()`, `strcat()`, `strlen()`, `isalpha()`.
4. Вывод: Исходное сообщение, сообщение после замены запрещённых слов, зашифрованное сообщение и количество замен.

Вариант 11. Тема: «Результаты экзамена»

Контекст: Преподаватель получает протокол экзамена в одной строке. Каждая запись содержит фамилию, группу и оценку: `Ivanov:IS-101:5`.

Техническое задание:

1. Ввод: Две строки – протокол экзамена и искомая оценка.
2. Действие: Разбить протокол на записи. Для каждой записи выделить фамилию, группу и оценку. Найти студентов с заданной оценкой. Сформировать строку с фамилиями и определить количество таких студентов. Дополнительно определить самый длинный идентификатор группы среди найденных студентов.
3. Функции: `strtok()`, `strchr()`, `strcmp()`, `strlen()`, `strcpy()/strcat()`.
4. Вывод: Исходный протокол, список студентов, количество найденных студентов и самый длинный идентификатор группы.

Вариант 12. Тема: «Анализ URL»

Контекст: Веб-сервер получает адрес страницы и должен определить основные компоненты URL перед обработкой запроса.

Техническое задание:

1. Ввод: Одна строка – URL.
2. Действие: Выделить протокол, домен и путь. Проверить, что протокол является *http* или *https*. Для *https* дополнительно определить соединение как защищённое. Найти

расширение страницы, если оно присутствует в пути. Сформировать краткую информацию об URL.

3. Функции: strstr(), strchr(), strcmp(), strcpy(), strcat(), strlen().
4. Вывод: Исходный URL, протокол, домен, путь, расширение страницы и статус соединения.

Контрольные вопросы

1. Что такое Си-строка? Чем она отличается от массива символов?
2. Что такое нуль-терминатор и зачем он необходим?
3. Чем отличается char *st от char st[N], где N – некоторая константа?
4. Как инициализировать строку и как ввести строку с клавиатуры?
5. Как определить длину строки?
6. Для чего используются strcpy() и strcat()?
7. Чем отличаются strcmp(), strchr() и strstr()?
8. Для чего используется strtok()? Почему после его применения необходимо учитывать изменение исходной строки?
9. Как функции isalpha(), isdigit(), isspace(), tolower() и toupper() помогают при обработке строк?
10. Что такое конкатенация строк?

Лабораторная работа №8. Структуры и Функции

Цель работы – изучение способов представления и обработки данных с использованием структур и вложенных структур в языке Си, получение навыков передачи структур и массивов структур в функции, а также освоение приёмов разработки функций в рамках заданного программного интерфейса.

Методические указания

При разработке программы необходимо использовать:

- **структуры** для представления объектов предметной области;
- **вложенную структуру** как поле основной структуры;
- **массив структур** для хранения нескольких объектов;
- **функцию ввода одной структуры**, передавая структуру по указателю;
- **функцию вывода массива структур** в табличном виде;
- **самостоятельно разработанную функцию** для изменения данных в соответствии с условием варианта;

- передачу данных в функции **через параметры**, без использования глобальных переменных;
- оператор **.** для обращения к полям структуры и оператор **->** для обращения к полям структуры через указатель.

Структура – составной тип данных, объединяющий несколько связанных между собой полей, которые могут иметь разные типы данных. Структуры позволяют представить в программе один объект предметной области как единое целое.

Вложенная структура – структура, являющаяся полем другой структуры. Она используется для объединения связанных между собой характеристик объекта. Например, дату можно представить отдельной структурой, содержащей день, месяц и год, и включить её в структуру студента.

Функция – именованный фрагмент программы, предназначенный для выполнения определённой задачи. Функция может получать данные через параметры и возвращать результат. Структуры могут передаваться в функции как по значению, так и по указателю. Массив структур также может передаваться в функцию вместе с количеством его элементов.

Лабораторное задание

Разработать программу на языке Си для обработки массива структур в соответствии с номером варианта.

Для каждого варианта необходимо самостоятельно спроектировать основную и вложенную структуры на основе описания предметной области.

Порядок выполнения работы

Порядок выполнения работы:

1. **Изучить шаблон программы**, приведённый в лабораторном задании.
2. **Спроектировать структуры данных** в соответствии с вариантом:
 - определить основную структуру;
 - определить вложенную структуру;
 - определить поля обеих структур и их типы;
 - включить вложенную структуру в состав основной структуры.
3. **Реализовать функцию ввода одной структуры** по заданной сигнатуре.
4. **Реализовать функцию вывода массива структур** в табличном виде по заданной сигнатуре.
5. **Разработать третью функцию самостоятельно** в соответствии с условием варианта:
 - определить имя функции;
 - определить необходимые параметры;
 - определить возвращаемый тип;

- реализовать требуемый алгоритм;
 - обеспечить изменение данных структур в соответствии с условием задачи.
6. **Написать функцию `main()`**, которая должна:
- создать массив структур;
 - организовать ввод данных;
 - вывести исходные данные;
 - вызвать самостоятельно разработанную функцию;
 - повторно вывести данные для проверки результата.
7. Выполнить программу на ЭВМ и убедиться в корректности полученных результатов.

Требования к реализации

Требования к реализации программы:

- Использовать структуры и вложенные структуры в соответствии с вариантом.
- Использовать массив структур для хранения нескольких объектов.
- Не использовать глобальные переменные для хранения данных.
- Функции должны работать с данными, переданными через параметры.
- Не изменять сигнатуры двух функций, заданных в шаблоне.
- Не изменять назначение готовых функций.
- Самостоятельная функция должна соответствовать условию варианта и изменять данные существующих структур.
- Не использовать динамическое выделение памяти.
- Результат работы самостоятельной функции должен быть продемонстрирован повторным выводом массива структур.

Общий шаблон программы

```
#include <stdio.h>

#include <string.h>


// =====

// ОПРЕДЕЛЕНИЕ ВЛОЖЕННОЙ СТРУКТУРЫ

// =====
```

```

// TODO: определить вложенную структуру
// в соответствии с вариантом

// =====

// ОПРЕДЕЛЕНИЕ ОСНОВНОЙ СТРУКТУРЫ
// =====

// TODO: определить основную структуру
// в соответствии с вариантом

// =====

// ФУНКЦИЯ ВВОДА ОДНОЙ СТРУКТУРЫ
// =====

// Параметры:
//   s - указатель на структуру.
//
// Назначение:
//   функция должна выполнить ввод всех данных
//   одной структуры.
//
// Сигнатуру функции изменять нельзя.

void inputName (Name *s)
{
    // TODO: написать код
}

// =====

```

```

// ФУНКЦИЯ ВЫВОДА МАССИВА СТРУКТУР
// =====

// Параметры:
//   arr - массив структур;
//   n   - количество элементов.
//
// Назначение:
//   функция должна вывести данные всех элементов
//   массива в виде таблицы.
//
// Сигнатуру функции изменять нельзя.

void printAllName(Name arr[], int n)
{
    // TODO: написать код
}

// =====

// ФУНКЦИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РЕАЛИЗАЦИИ
// =====

// Здесь необходимо самостоятельно:
// 1. Объявить функцию.
// 2. Определить её параметры.
// 3. Определить возвращаемый тип, если это необходимо.
// 4. Реализовать заданный алгоритм.
// 5. Изменить данные в соответствии с условием варианта.
//
// TODO: написать функцию

```

```

// =====

// ФУНКЦИЯ MAIN

// =====

int main()
{
    // TODO: создать массив структур

    // TODO: ввести данные

    // TODO: вывести исходные данные

    // TODO: вызвать самостоятельно разработанную функцию

    // TODO: вывести изменённые данные

    return 0;
}

```

Примечание: *Name* в шаблоне является условным обозначением. При выполнении работы необходимо заменить его на имя основной структуры, выбранное в соответствии с вариантом. Аналогично необходимо самостоятельно определить имя вложенной структуры и названия её полей.

Вариант 1. Тема: «Заявки сервисного центра»

Контекст: Сервисный центр хранит сведения о заявках на ремонт техники. Для каждой заявки необходимо хранить номер заявки, название устройства, стоимость ремонта и текущий статус. Также необходимо хранить сведения о клиенте: имя и номер телефона.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Перевести все заявки со статусом «принята» в статус «в работе», если стоимость ремонта больше нуля.

Вариант 2. Тема: «Студенты учебной группы»

Контекст: Учебная группа хранит сведения о студентах. Для каждого студента необходимо хранить имя, учебную группу, средний балл и размер стипендии. Кроме того, необходимо хранить дату рождения студента: день, месяц и год.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для студентов со средним баллом не менее 4,5 увеличить размер стипендии на 20%.

Вариант 3. Тема: «Заказы интернет-магазина»

Контекст: Интернет-магазин хранит сведения о заказах. Для каждого заказа необходимо хранить его номер, название товара, количество единиц товара, стоимость и текущий статус. Также необходимо хранить сведения о покупателе: имя и номер телефона.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Перевести заказы со статусом «оформлен» в статус «подготовка к отправке», если их стоимость превышает 5000 рублей.

Вариант 4. Тема: «Книги библиотеки»

Контекст: Библиотека хранит сведения о книгах. Для каждой книги необходимо хранить название, автора, количество страниц и состояние экземпляра. Также необходимо хранить сведения об издании: название издательства и год выпуска.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для книг, выпущенных более 10 лет назад, уменьшить стоимость на 20%.

Вариант 5. Тема: «Автомобили автосервиса»

Контекст: Автосервис хранит сведения об автомобилях, находящихся на обслуживании. Для каждого автомобиля необходимо хранить марку, модель, стоимость обслуживания и текущий статус. Также необходимо хранить характеристики двигателя: объём и мощность.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для автомобилей с мощностью двигателя более 150 л. с. установить тип обслуживания «расширенное».

Вариант 6. Тема: «Сотрудники организации»

Контекст: Организация хранит сведения о сотрудниках. Для каждого сотрудника необходимо хранить имя, должность, заработную плату и размер надбавки. Также необходимо хранить дату приёма на работу: день, месяц и год.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для сотрудников, принятых на работу до 2020 года, установить надбавку в размере 10% от текущей заработной платы.

Вариант 7. Тема: «Авиарейсы»

Контекст: Авиакомпания хранит сведения о рейсах. Для каждого рейса необходимо хранить номер, пункт назначения, стоимость билета, количество свободных мест и статус. Время отправления задаётся часами и минутами.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для рейсов, на которых осталось менее 20 свободных мест, изменить статус с «продажа билетов» на «ограниченная продажа».

Вариант 8. Тема: «Спортсмены»

Контекст: Спортивный клуб хранит сведения о спортсменах. Для каждого спортсмена необходимо хранить имя, вид спорта, возраст и рейтинг. Также необходимо хранить сведения о тренере: имя и стаж работы.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для спортсменов с рейтингом не менее 80 установить спортивный статус «кандидат на участие в соревнованиях».

Вариант 9. Тема: «Номера гостиницы»

Контекст: Гостиница хранит сведения о номерах. Для каждого номера необходимо хранить номер комнаты, категорию, стоимость проживания, количество мест и текущий статус. Также необходимо хранить сведения о расположении номера: корпус и этаж.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для свободных номеров, рассчитанных минимум на трёх человек, установить категорию «семейный».

Вариант 10. Тема: «Учебные курсы»

Контекст: Университет хранит сведения об учебных курсах. Для каждого курса необходимо хранить название, количество учебных часов, количество студентов и текущий статус. Также необходимо хранить сведения о преподавателе: имя и стаж преподавания.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для курсов, на которых обучается более 50 студентов, увеличить количество учебных часов на 2 часа.

Вариант 11. Тема: «Товары на складе»

Контекст: Склад хранит сведения о товарах. Для каждого товара необходимо хранить название, категорию, цену, количество на складе и статус. Также необходимо хранить сведения о производителе: название компании и страну производства.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для товаров, которых осталось менее 5 единиц, установить статус «заканчивается».

Вариант 12. Тема: «Фильмы»

Контекст: Кинопрокатная компания хранит сведения о фильмах. Для каждого фильма необходимо хранить название, жанр, рейтинг и продолжительность. Также необходимо хранить дату выхода фильма: день, месяц и год.

Структуру программы взять из лабораторного шаблона. Структуры данных спроектировать самостоятельно по описанию.

Задание для самостоятельной функции:

Для фильмов с рейтингом не менее 8,5 установить статус «рекомендован к просмотру».

Контрольные вопросы

1. Что такое структурный тип данных и для каких задач он используется в языке Си?
2. Что такое вложенная структура? Для каких данных целесообразно использовать вложенные структуры?
3. Каким образом осуществляется обращение к полям структуры? Чем отличаются операторы `.` и `->`?
4. Как объявить массив структур и как получить доступ к полям отдельного элемента массива?
5. Что такое функция? Для чего функции используются при разработке программ?

6. Что такое прототип функции и из каких частей он состоит? Чем прототип отличается от вызова и определения функции?
7. Чем отличается передача структуры в функцию по значению от передачи по указателю? В каком случае следует использовать указатель?
8. Как передаётся массив структур в функцию? Для чего функции дополнительно передаётся количество элементов массива?
9. Какие требования предъявляются к формальным и фактическим параметрам функции? Что произойдёт, если их типы или количество не соответствуют друг другу?
10. Как функция может изменить поля структуры, переданной ей в качестве параметра? Почему для этого может потребоваться передача структуры по указателю?

Лабораторная работа №9. Динамические структуры данных

Цель работы – изучение способов организации и обработки односвязных динамических списков в языке Си, получение навыков работы с динамической памятью, указателями и узлами списка.

Методические указания

Динамический список – структура данных, количество элементов которой может изменяться во время выполнения программы.

Односвязный список состоит из узлов. Каждый узел содержит информационную часть и указатель на следующий узел списка. Последний узел содержит указатель NULL.

Двусвязный список состоит из узлов, каждый из которых содержит информационную часть и два указателя: на следующий и предыдущий узлы списка. Это позволяет перемещаться по списку как в прямом, так и в обратном направлении.

Для работы с **динамической памятью** используются функции:

- *malloc()* – выделение памяти для нового узла;
- *free()* – освобождение выделенной памяти.

При обходе списка необходимо последовательно переходить от одного узла к следующему до достижения NULL.

При разработке программы необходимо использовать:

- односвязный динамический список;
- структуры и вложенную структуру из лабораторной работы №8;
- динамическое выделение памяти для узлов списка;
- функции для создания, обработки и освобождения элементов списка;
- самостоятельную функцию обработки данных, заданную вариантом лабораторной работы №8.

Лабораторное задание

Разработать программу на языке Си для обработки массива структур в соответствии с номером варианта. Данная лабораторная работа выполняется на основе выбранного варианта **лабораторной работы №8**:

1. Сохранить основную и вложенную структуры, разработанные в лабораторной работе №8.
2. Представить набор объектов в виде **односвязного динамического списка**. Каждый узел должен содержать данные одного объекта и указатель на следующий узел.
3. Разработать программу, осуществляющую:
 - ввод элементов списка до команды завершения ввода, в списке должно быть не менее **пяти** элементов;
 - вывод исходного списка в виде таблицы;
 - реализацию самостоятельной функции из выбранного варианта лабораторной работы №8, адаптированной для работы с динамическим списком;
 - вывод изменённого списка в виде таблицы;
 - освобождение памяти, выделенной для элементов списка.
4. Выполнить программу на ЭВМ и убедиться в корректности полученных результатов.

Контрольные вопросы

1. Что такое динамическая структура данных и чем она отличается от массива?
2. Что представляет собой односвязный список? Из каких частей состоит его элемент?
3. Чем односвязный список отличается от двусвязного списка?
4. Что такое указатель на следующий элемент списка и какое значение хранится в нём у последнего элемента?
5. Для чего используются функции `malloc()` и `free()` при работе с динамическими структурами?
6. Как осуществляется добавление нового элемента в односвязный список?
7. Как осуществляется последовательный просмотр элементов односвязного списка?
8. Почему при работе с динамической памятью необходимо освобождать память после завершения работы со списком?
9. Чем обработка массива структур отличается от обработки односвязного списка?
10. Как изменить функцию обработки массива структур так, чтобы она выполняла тот же алгоритм для элементов односвязного списка?

Лабораторная работа №10. Файлы и Многомодульные программы

Цель работы – изучение принципов работы с файлами и организации многомодульных программ на языке Си, получение навыков сохранения и восстановления данных динамических структур из файла и разделения программы на отдельные программные модули.

Методические указания

Файл – именованная область внешней памяти, предназначенная для долговременного хранения данных. Для работы с файлами в языке Си используется тип **FILE *** и функции библиотеки **stdio.h**.

Перед выполнением операций файл необходимо открыть, а после завершения работы – закрыть. Основные режимы открытия файлов:

- *r* – чтение;
- *w* – запись;
- *a* – добавление данных в конец файла.

При открытии файла необходимо проверить результат операции. Для закрытия файла используется функция **fclose()**.

Для чтения и записи текстовой информации могут использоваться функции **fscanf()**, **fprintf()**, **fgets()**, **fputs()** и другие функции стандартной библиотеки.

Многомодульная программа состоит из нескольких исходных файлов, содержащих логически связанные части программы. Разделение программы на модули позволяет организовать код и упростить его разработку и сопровождение.

Для объявления структур, констант и прототипов функций, используемых несколькими модулями, применяются заголовочные файлы с расширением **.h**. Заголовочный файл подключается к исходному файлу с помощью директивы **#include**.

При разработке программы рекомендуется выделять в отдельные модули функции, относящиеся к различным задачам программы, например функции работы со списком и функции работы с файлами.

Лабораторное задание

Данная лабораторная работа выполняется на основе выбранного варианта **лабораторной работы №9**:

1. Сохранить предметную область, основную и вложенную структуры, разработанные в лабораторной работе №9.
2. Представить данные в виде односвязного динамического списка.
3. Разработать программу, которая выполняет:
 - ввод исходных данных и формирование динамического списка;

- вывод элементов списка в виде таблицы;
- сохранение данных списка в файл;
- чтение данных из файла и восстановление динамического списка;
- выполнение самостоятельной функции обработки данных, заданной вариантом;
- вывод изменённого списка в виде таблицы;
- сохранение результатов обработки в файл;
- освобождение выделенной динамической памяти.

4. Разделить программу на несколько логически связанных модулей. Для общих структур и прототипов функций рекомендуется использовать отдельный заголовочный файл.

5. Выполнить программу на ЭВМ и убедиться в корректности полученных результатов.

Примечание: При сохранении списка в файл необходимо записывать информационные данные элементов, а не указатели, используемые для связи узлов списка. При чтении из файла на основании сохранённых данных должен быть заново сформирован динамический список.

Например, данные нескольких элементов могут быть представлены в файле следующим образом:

Ноутбук;75000;3;в наличии

Смартфон;45000;8;в наличии

Планшет;32000;2;заканчивается

Конкретный формат хранения данных и способ разделения полей определить самостоятельно.

Контрольные вопросы

1. Что такое файл и для чего он используется в программе?
2. Какие основные режимы открытия файла существуют в языке Си? Для чего используются режимы r, w и a?
3. Какие функции языка Си используются для открытия, закрытия, чтения и записи файлов?
4. Как проверить, что файл успешно открыт? Что необходимо сделать, если открыть файл не удалось?
5. Что такое многомодульная программа? Для чего программу разделяют на несколько исходных файлов?
6. Для чего используется заголовочный файл .h и что обычно в него помещают?
7. Как осуществляется взаимодействие между модулями многомодульной программы?